

Аннотация

Тема работы — веб-приложение для подготовки к техническим собеседованиям в рамках стартапа Viewtrain. Разрабатывается клиентская часть. Приложение использует GigaChat в роли AI-интервьюера. От фронтенда требуется три вещи: поддерживать диалог с нейросетью, дать пользователю удобный редактор кода для решения алгоритмических задач и после собеседования показать детальный разбор ответов.

Перед началом разработки были изучены типичные сценарии прохождения технических собеседований — какие вопросы задают, как кандидаты отвечают. Также проведён обзор интерфейсов конкурентных платформ (LeetCode, Pramp, InterviewBit). Выявлено, что их интерфейсы часто перегружены либо, наоборот, в них не хватает важных элементов. На основе этого сформулированы требования к клиентской части.

Для реализации выбран Vue.js. Причина — понятная документация и хорошая совместимость с задачами проекта. Управление состоянием организовано через Zustand (проще и легче, чем классический Vuex). Маршрутизация — на Vue Router. В итоге архитектура получилась модульной: каждый блок можно менять независимо.

Реализованы три основных модуля. Первый модуль — это авторизация (вход и регистрация) с клиентской валидацией данных, чтобы сократить количество неверных запросов к серверу. Второй модуль это дашборд с графиками, где пользователь видит свою статистику и может быстро начать новую сессию. Третий модуль — это главный экран собеседования. На нём работает чат, причём ответы AI передаются не целиком, а частями. Такой подход выбран потому, что при мгновенном выводе ответа теряется эффект живого диалога, а это важно для тренировки.

Панель ввода на экране собеседования адаптируется под тип вопроса. Для алгоритмических задач используется Monaco Editor тот же редактор, что в VS Code, с подсветкой синтаксиса. Выбор пал на Monaco, а не на CodeMirror, из-за большей мощности и удобства интерфейса для разработчиков. Для

обычных текстовых ответов используется стандартное поле ввода. Таймер и шкала прогресса вынесены на боковую панель, чтобы постоянно находиться в поле зрения.

В работе приведено описание интерфейса. Перечень экранов, схема навигации между ними, состав отображаемых данных. Представлена компонентная схема фронтенда (чат, редактор кода, панель таймера). Обычные запросы идут через REST API, для стриминга ответов AI задействован WebSocket, потому что REST неудобен для передачи потока токенов. WebSocket позволяет отправлять данные частями с немедленной отрисовкой на экране. В итоге получена работающая клиентская часть, пригодная для использования в качестве основы коммерческого продукта Viewtrain. Все ключевые функции реализованы. Диалог с AI, редактор кода, формирование отчётов. Для запуска стартапа потребуется добавить платёжный модуль и маркетинговую инфраструктуру. Кроме того, проект может применяться в учебных целях, как пример веб-интерфейса с реактивностью, стримингом и модульной архитектурой.

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	3
ВВЕДЕНИЕ	6
1. ТЕОРЕТИКО-АНАЛИТИЧЕСКАЯ ЧАСТЬ	8
1.1 Обзор зарубежных исследований	8
1.2 Обзор отечественных исследований.....	11
1.3 Анализ существующих решений на рынке.....	12
1.4 Общая характеристика проблемы исследования.....	14
1.5 Основные методы решения проблем	16
1.6 Основные технологии решения проблемы	18
1.7 Сравнительный анализ frontend-фреймворков и архитектурных подходов.....	22
1.8 Алгоритм выбора методов и технологий.....	24
1.9 Выводы по теоретико-аналитической части.....	27
2. ПРАКТИЧЕСКАЯ ЧАСТЬ СТАРТАП-ПРОЕКТА VIEWTRAIN	29
2.1 Описание стартап-проекта ViewTrain	29
2.1.1 Описание проекта и продукта	29
2.2 Реализация клиентской части	31
2.2.1 Этапы разработки	31
2.2.2 Компонентная архитектура на Vue	32
2.2.3 Модуль диалога с AI-интервьюером	32
2.2.4 Редактор кода и панель ввода	33
2.2.5 Модуль аналитики и визуализации прогресса	34
2.2.6 Интеграция с GigaChat API	35
2.2.7 Управление состоянием через Pinia.....	35

2.2.8 Маршрутизация и защита маршрутов	36
2.2.9 Тестирование и валидация.....	37
2.2.10 Перспективы развития	37
2.2.11 Принципы UX-дизайна для стрессовых сценариев	37
2.2.12 Информационная архитектура и навигация	38
2.2.13 Описание ключевых экранов	38
2.2.14 Выбор UI-библиотеки и компонентов.....	39
2.2.15 Модель данных клиентской части	39
2.2.16 Реактивность и управление состоянием	39
2.2.17 Адаптивность и доступность интерфейса	40
2.2.18 Экран авторизации и регистрации	40
2.2.19 Экран результатов сессии.....	42
2.2.20 Дашборд и статистика.....	43
2.3. Экономическое обоснование	45
2.3.1 Затраты на разработку	45
2.3.2 Операционные расходы	45
2.3.3 Модель монетизации и ценообразование	45
2.3.4 Прогноз окупаемости	45
ЗАКЛЮЧЕНИЕ.....	48
СПИСОК ЛИТЕРАТУРЫ.....	49
ПРИЛОЖЕНИЯ	51
Приложение А — Структура файлов проекта.....	51
Приложение Б — Ключевые фрагменты кода.....	52
Б.1 Конфигурация API-эндпоинтов (config.js).....	52
Б.2 HTTP-клиент с авторизацией (client.js)	53

Б.3 Модуль аутентификации (auth.js).....	54
Б.4 Компонент чата собеседования (ChatInterview.jsx).....	55
Б.5 Маршрутизация приложения (App.jsx)	56
Приложение В — API-контракты	57

ВВЕДЕНИЕ

С каждым годом рост требований к уровню подготовки IT-специалистов для прохождения успешного собеседования возрастает. Всё большую актуальность приобретают цифровые инструменты, позволяющие отрабатывать навыки прохождения технических собеседований в интерактивном формате. В связи с этим увеличивается интерес к программным решениям, способным имитировать реальный процесс собеседования и обеспечивать пользователю регулярную практику. Одним из таких решений является ViewTrain.

Это веб-приложение, которое помогает подготовиться к техническим собеседованиям. Диалог проходит с AI-интервьюером на базе GigaChat. Для начала пользователю нужно выбрать тип интервью и уровень сложности. После чего происходит симуляция живого диалога. В результате пользователь приложения получает структурированную обратную связь по каждому ответу. Также ViewTrain позволяет отслеживать динамику прогресса в личном кабинете.

ViewTrain реализуется в формате технологического стартапа в сфере EdTech. Целевая аудитория продукта — IT-специалисты уровней junior и middle, готовящиеся к техническим собеседованиям. Проблема целевой аудитории заключается в том, что занятия с менторами стоят дорого и не обеспечивают регулярности, платформы алгоритмических задач не дают опыта живого диалога с интервьюером, а сервисы парных тренировочных интервью зависят от наличия второго участника. ViewTrain закрывает этот пробел, предоставляя неограниченную практику диалоговых интервью с AI-интервьюером и автоматической обратной связью. На рынке отсутствуют прямые аналоги, сочетающие диалоговый формат, использование отечественной языковой модели GigaChat и ориентацию на специфику технических собеседований в IT, что создаёт потенциал для коммерциализации и масштабирования проекта.

Цель работы: спроектировать и реализовать клиентскую часть платформы, которая обеспечит удобное взаимодействие пользователя с AI-интервьюером. Для достижения цели решались следующие задачи: анализ существующих платформ и исследований в области адаптивного обучения, выбор технологического стека, проектирование интерфейса под стрессовый сценарий собеседования, реализация модулей чата, редактора кода и аналитики, тестирование.

1. ТЕОРЕТИКО-АНАЛИТИЧЕСКАЯ ЧАСТЬ

1.1 Обзор зарубежных исследований

Зарубежные исследования по теме цифрового обучения и оценки ИТ-специалистов ведутся давно, но последние лет пять их стало заметно больше, потому что ИТ-компаниям остро не хватает разработчиков, а собеседования усложнились до такой степени что без подготовки устроиться почти нереально. В рамках данной работы рассматривались публикации за 2021–2024 годы. Больше всего релевантного материала нашлось на ACM CHI и EDM, остальные конференции (ACM SIGCSE, IEEE EDUCON, AIED) тоже дали материал, но в меньшем объёме. Разброс тем большой. На CHI в 2022 году, например, показали, что даже такая мелочь как анимация при правильном ответе заметно влияет на то вернётся пользователь завтра или нет. А на EDM и AIED исследуют вещи посерьёзнее, в частности как по поведению обучающегося заранее понять, что он застрянет на следующей теме.

Главная идея, которая проходит через многие работы — обучение должно подстраиваться под конкретного человека. Если двое решают одну задачу, но один справляется за минуту, а другой за час, им явно нужно давать разные следующие задания.

Формально эта идея оформилась ещё в 1997 году, когда Koedinger с соавторами предложили модель Knowledge Tracing. Суть простая: система смотрит как ученик решает задачи, где ошибается, когда просит подсказку, и на основе этого пересчитывает вероятность того, что он освоил тему. Математика байесовская. Позже появился Deep Knowledge Tracing, где вместо байесовских сетей поставили нейросети. Точность выросла, но и данных для обучения стало нужно в разы больше. Самый известный пример применения это Cognitive Tutor из Карнеги-Меллон, его разрабатывали больше двадцати лет и исследования показали, что результаты пользователей на 15–20% лучше чем у тех, кто учился по обычным учебникам. Другой подход реализован в

ALEKS, там используется теория пространств знаний и система строит карту освоенных тем. В американских школах ALEKS используют довольно активно. Без проблем тоже не обошлось. Для хорошей работы нужно много данных, а когда появляется новая тема система первое время работает вслепую, это называют проблемой холодного старта. Нейросетевые модели часто не объясняют свои рекомендации, и пользователь их игнорирует. Плюс модель, обученная на американских школьниках, может не сработать на российских студентах, у них другая база и другие ошибки, а дообучение стоит дорого.

Раньше AI в образовании использовали в основном для подсчётов, сколько ошибок, какие темы сложнее. С появлением GPT-3, GPT-4, Claude и других больших языковых моделей ситуация изменилась принципиально. AI научился сам генерировать учебный контент, не по шаблону, а под конкретного ученика. В 2023 году Kasneci с коллегами опубликовали обзор, в котором систематизировали возможности LLM в образовании, от генерации персонализированных заданий и объяснения сложных тем простым языком до ведения полноценного диалога в роли тьютора и оценки развёрнутых ответов. На основе этих возможностей начали строить Intelligent Tutoring Systems, то есть системы, которые реально разговаривают с учеником, а не просто выдают задания. Параллельно с этим Deterding с коллегами ещё в 2011 году написали базовую работу по геймификации в обучении и выделили очки, уровни, бейджи, рейтинги и сюжет как ключевые элементы. Правда потом выяснилось, что работает это не всегда. Рейтинги могут демотивировать тех, кто внизу, а бейджи, которые сыплются, слишком часто быстро обесцениваются. Самые интересные результаты получаются, когда AI и геймификация работают вместе. AI подбирает сложность так чтобы ученик оставался в состоянии потока, а игровые механики подкрепляют успехи.

«Автоматизировать получается далеко не всё. Код и алгоритмы проверять тестами научились, а вот с soft skills у AI пока плохо. На собеседовании спрашивают про опыт, про работу в команде, про конфликты, и

попытки оценивать это автоматически через анализ видео и тона голоса дают низкую точность. Модель легко принимает волнение за неуверенность, а паузу за незнание. Это не единственная сложность. Пользователь хочет понимать почему ему поставили такую оценку, а модель молчит. Мехраби с коллегами в 2021 году показали, что тут ещё и bias накладывается, модель, обученная на данных конкретной компании копирует её предубеждения по полу, возрасту, образованию. Лечится это тяжело, нужно вручную чистить выборку и потом перепроверять. В HRTech полноценное поведенческое интервью на базе AI называют святым Граалем, и пока до него далеко.

Основные направления зарубежных исследований, рассмотренные в разделе 1.1, можно представить в виде схемы (рисунок 1.2). Адаптивные системы и AI/LLM в образовании развиваются параллельно и всё чаще пересекаются, геймификация добавляет к ним механики вовлечения. В нижней части схемы выделены два вызова которые пока не имеют удовлетворительного решения, оценка soft skills и этические проблемы. Именно они остаются в фокусе научных дискуссий.

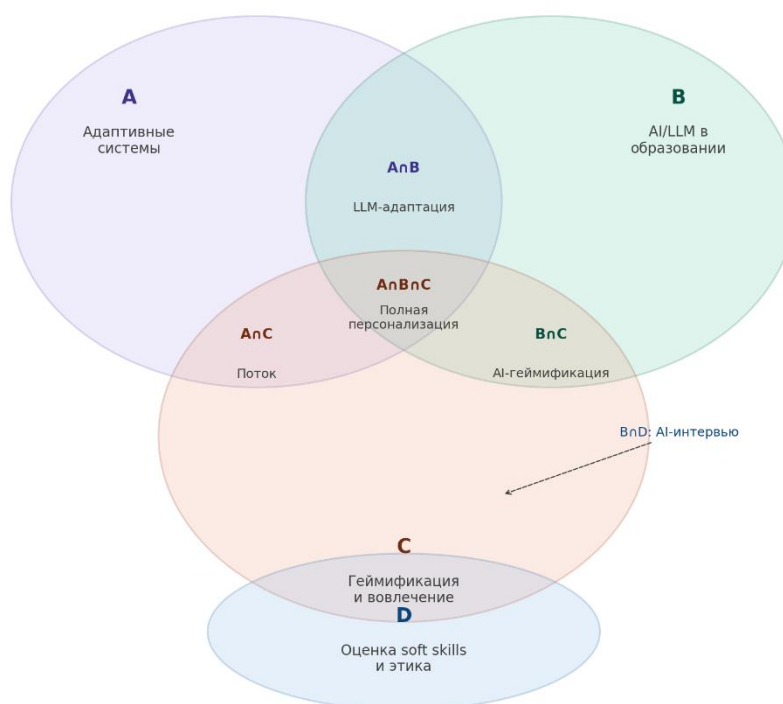


Рисунок 1.2 — Ключевые направления зарубежных исследований

1.2 Обзор отечественных исследований

В России этой темой тоже занимаются. У нас в МИРЭА на кафедре индустриального программирования исследуют автоматическую проверку кода и системы подготовки к собеседованиям. Из крупных центров стоит выделить НИУ ВШЭ, где в 2022 году изучали как цифровые следы студентов связаны с успеваемостью.

Российские исследования в этой области отличаются от зарубежных нескольких вещей. Во-первых, многие работы мотивированы импортозамещением, нужно делать свои аналоги LMS, систем прокторинга, платформ для подготовки к собеседованиям, которые могут работать без зарубежных сервисов. Во-вторых, в российских компаниях требования к кандидатам другие. Больше внимания уделяется знанию конкретных фреймворков популярных у нас или умению работать с 1С, и исследователи пытаются понять, как это влияет на процесс подготовки. В фундаментальных исследованиях по адаптивным системам мы пока отстаём, потому что там нужны большие вложения в данные и разработку. Зато в прикладных вещах есть свои результаты, например методики построения индивидуальных траекторий для конкретных специальностей, анализ типичных ошибок на собеседованиях в российские компании, симуляторы для отработки навыков работы с отечественным стеком.

Из российских продуктов стоит упомянуть Яндекс.Практикум и Хекслет. Яндекс.Практикум — это коммерческая платформа, но они публикуют статьи о проектировании учебного опыта и о том какие механики удержания студентов реально работают. Хекслет интересен тем, что у них открытый код и много публичной статистики по прохождению курсов, видно на каких темах студенты застревают чаще всего. В некоторых вузах тоже есть свои платформы, например в МФТИ разрабатывают систему автоматической проверки кода, которая даёт развёрнутую обратную связь, а не просто говорит принято или нет.



Рисунок 1.2.4 — Карта российских исследовательских центров
Российские исследовательские центры

1.3 Анализ существующих решений на рынке

LeetCode запустили в 2015 году и с тех пор она стала главной площадкой для тех, кто готовится к алгоритмическим собеседованиям. Больше 2000 задач, разбитых по темам и сложности, встроенный редактор с тестами, живое сообщество, которое постоянно пишет разборы и делится решениями. Всё это работает и реально помогает прокачать алгоритмы. Только вот готовишься ты в тишине. Сидишь один, пишешь код, никто не спросит почему именно такой подход и что будет если данных станет в сто раз больше. На настоящем собеседовании половина успеха это диалог, умение объяснять ход мысли, и именно этого на LeetCode нет. Системный дизайн добавили недавно, сделан пока слабо. Поведенческих вопросов нет вообще.

Pramp работает совсем иначе. Никаких задач из базы. Два человека созваниваются и по очереди собеседуют друг друга, есть редактор кода и доска

для схем. Ощущения максимально близкие к настоящему интервью, потому что надо думать вслух и укладываться вовремя. Звучит отлично, пока не столкнёшься с реальностью. Партнёра подбирает система, и повлиять на это нельзя. Кто-то задаёт хорошие вопросы, кто-то читает условие с листочка и молчит. Бывает, что человек просто не приходит. Расписание тоже надо согласовывать. AI-оценки нет, обратную связь даёт тот же партнёр

InterviewBit устроен иначе. Там не просто задачи, а полноценный курс с темами и рейтингом. Рейтинг считается по количеству решённых задач и скорости. Фишка в том, что компании-партнёры видят профили лучших пользователей и сами зовут их на собеседования. Для тех, кто не знает с чего начать это удобно, потому что путь уже выстроен. Правда сам процесс всё равно одиночный. Сидишь, решаешь задачи, никакого диалога. Поведенческие вопросы и системный дизайн почти не затронуты.

AlgoExpert пошёл другим путём и сделал ставку на объяснения. Каждую задачу разбирает эксперт на видео, причём не просто показывает решение, а объясняет почему именно так, какая сложность и где можно оптимизировать. Есть отдельные модули по системному дизайну (SystemsExpert) и поведенческим вопросам (BehavioralExpert). По качеству разборов это, пожалуй, лучшее что есть на рынке. Проблема в другом. Пользователь смотрит видео и на этом всё заканчивается. Написать своё решение и получить на него обратную связь нигде.

В России есть несколько небольших платформ под локальный рынок. Они готовят именно к тому, что спрашивают в наших компаниях: 1С, отечественные CRM, специфические стеки. Только вот сделаны эти платформы, как правило, на коленке. Интерфейсы неудобные, адаптивности нет, про AI речи не идёт. Зашёл, порешал пару задач и забыл.

Если смотреть на все платформы вместе, видна закономерность: каждая закрывает только одну потребность. LeetCode даёт задачи. Pramp даёт диалог. InterviewBit и AlgoExpert дают структуру и объяснения.

Таблица 1.1 — Сравнительный анализ платформ для подготовки к собеседованиям

Критерий	LeetCode	Pramp	InterviewBit	AlgoExpert
Алгоритмические задачи	2000+	Нет	Есть	Есть
Диалог с интервьюером	Нет	Да (живой)	Нет	Нет
Системный дизайн	Слабо	Есть	Нет	Да
Поведенческие вопросы	Нет	Есть	Нет	Да
Адаптивность под пользователя	Нет	Нет	Частично	Нет
Обратная связь	Тесты	От партнера	Рейтинг	Видео разбор
AI-интервьюер	Нет	Нет	Нет	Нет
Редактор кода	Да	Да	Да	Нет
Стоимость	\$35/месяц	Бесплатно	Бесплатно/ Scaler \$3000	\$99/год

Ни одна из рассмотренных платформ не даёт всего сразу. Задачи есть, но они не генерируются под конкретного человека, а просто берутся из общей базы. Диалог есть, но только там, где он зависит от случайного партнёра. Аналитика есть, но останавливается на процентах и счётчиках, не объясняя, что именно повторить и почему. Всё это вместе не собрал никто, и именно здесь есть место для платформы с AI-интервьюером на базе GigaChat.

1.4 Общая характеристика проблемы исследования

Проведённый анализ выявил характерное противоречие: научное сообщество накопило достаточно знаний о том, как строить адаптивные образовательные системы, но до реального продукта эти знания практически не доходят.

Зарубежные исследователи эту проблему проработали достаточно глубоко. Модели Knowledge Tracing и Deep Knowledge Tracing давно вышли за рамки академических статей их тестировали на реальных учебных системах. Байесовские подходы к персонализации, применение LLM для генерации контента и ведения учебного диалога, механики геймификации всё это не

гипотезы, а вещи, которые уже проверены на практике и описаны с конкретными результатами.

Практика расходится с теорией. Существующие платформы для подготовки к собеседованиям развивались органически, каждая вокруг своей одной идеи, и в итоге рынок получился фрагментированным. Где есть задачи, нет диалога. Где есть диалог, нет гарантии его качества. Где хорошо объясняют, там пользователь пассивен. Про то, чтобы система подстраивалась под конкретного человека, а не просто предлагала выбрать уровень сложности, речи не идёт нигде.

В России ситуация своя. Российские платформы понимают локальный контекст: знают, что спрашивают в наших компаниях, какие технологии востребованы, где кандидаты чаще всего спотыкаются. Только этого недостаточно, если интерфейс сделан десять лет назад, адаптивности нет никакой, а вся аналитика — это счётчик решённых задач. Зарубежные инструменты в этом смысле сделаны лучше, но они проектировались под другой рынок и другие реалии найма.

Здесь и возникает вопрос, которому посвящена эта работа. Пользователю нужен не набор инструментов, а цельный путь подготовки. Сейчас такого пути нет либо платформа знает российскую специфику, но сделана слабо, либо сделана хорошо, но не про наш рынок. Про персонализацию и живой диалог с обратной связью речи не идёт нигде.

В данной работе разрабатывается клиентская часть такой платформы. Backend и языковая модель выступают как внешние сервисы, с которыми интерфейс взаимодействует через API. Задача интерфейса — не перегружать пользователя в и без того стрессовой ситуации, прозрачно отображать все состояния системы, предоставлять удобные инструменты под разные форматы ответов и наглядно показывать прогресс. Всё это требует отдельного проектного решения, которому посвящены следующие разделы.

1.5 Основные методы решения проблем

При разработке платформы для подготовки к собеседованиям можно использовать разные подходы. Прежде чем выбирать конкретный, имеет смысл разобраться какие методы вообще существуют и что каждый из них даёт на практике.

Самый старый и простой подход это курирование контента. Принцип такой: собрать материалы, разложить по полочкам и отдать пользователю. Берётся экспертное знание о том какие вопросы задают на собеседованиях, формируется база задач и теоретических материалов, потом всё это структурируется по темам, сложности и типам собеседований. Пользователь заходит, выбирает что хочет учить и решает. Именно так работают LeetCode, InterviewBit, AlgoExpert. Подход понятный, дешёвый в разработке и прозрачный, никакой магии, вот задача, вот тесты. Проблема в том, что он статичный. Все пользователи видят одно и то же независимо от уровня. Система не знает, что ты уже решил 50 задач на динамику и продолжает подсовывать те же темы. Если решаешь быстро она не ускоряется, если медленно не предлагает повторений. Для быстрого старта и разнородной аудитории этого хватает, но для эффективного обучения нет.

Адаптивный подход устроен сложнее. Система следит за тем, как пользователь решает задачи и на основе этого меняет траекторию. Решил быстро и правильно, следующая задача будет сложнее. Застрял, система откатывается назад и предлагает повторение. Технически это реализуется через модели Knowledge Tracing или их нейросетевые варианты. Плюс очевиден, каждый пользователь идёт по своему пути и не тратит время на то, что уже знает. Минусов тоже хватает. Для обучения модели нужно много данных, на старте их нет, и система работает вслепую. Реализация дорогая, нужен отдельный ML-пайплайн. И самое главное, адаптивность работает только с задачами, у которых есть чёткий ответ. Оценить открытый текстовый ответ или качество диалога классический адаптивный алгоритм не может.

Симуляция это попытка воспроизвести реальную ситуацию собеседования. Пользователь не просто решает задачу, а общается с интервьюером, объясняет ход мысли, отвечает на уточняющие вопросы. Раньше это делалось только через живых людей, как на Pramp. Сейчас появилась возможность заменить человека на LLM. Преимущество в том что симуляция тренирует именно то, что нужно на реальном интервью, умение думать вслух и работать под давлением. Проблема в оценке. Когда ответ — это код, проверить его легко, запустил тесты и всё понятно. Когда ответ — это рассказ про опыт работы в команде, объективно оценить его гораздо сложнее. LLM справляются с этим лучше, чем rule-based системы, но до человеческого уровня оценки ещё далеко. Плюс симуляция дорогая в реализации, каждый запрос к LLM стоит денег и добавляет задержку.

Ни один метод в одиночку не решает проблему полностью. Курирование контента даёт базу, но не персонализацию, адаптивные системы персонализируют, но не учат диалогу, симуляторы учат диалогу, но сложны в оценке. Современный подход в том, чтобы комбинировать все три. Нужна база контента, из которой можно выбирать, нужна адаптивность чтобы подстраиваться под конкретного пользователя, и нужна симуляция чтобы тренировать диалог. Для данной работы это означает следующее: берём готовую базу знаний или генерируем её с помощью AI, подстраиваем сложность под пользователя и делаем это в формате живого диалога. Как это реализовать технически разбирается в следующих разделах.

На рисунке 1.1 показано как три метода подготовки к собеседованиям соотносятся между собой по трём параметрам: персонализация, реалистичность диалога и простота реализации. Курирование контента проще всего внедрить, но оно не подстраивается под пользователя и не тренирует диалог. Адаптивные системы хорошо персонализируют процесс, средние по сложности, но диалога в них тоже нет. Симуляция даёт максимально реалистичный опыт, зато реализовать её сложнее всего. Ни один метод не

закрывает все три потребности, что и обосновывает необходимость их комбинации



Рисунок 1.1 — Сравнение трёх основных методов подготовки к собеседованиям

1.6 Основные технологии решения проблемы

Теперь посмотрим, с помощью каких технологий можно реализовать описанные методы.

Для адаптивности нужны модели машинного обучения, и здесь стандартный выбор это Python с его экосистемой. Основные библиотеки для работы с данными это Pandas и NumPy, для классических моделей вроде логистической регрессии и случайного леса используется Scikit-learn, для градиентного бустинга на табличных данных LightGBM и XGBoost. Если делать Deep Knowledge Tracing или анализировать тексты ответов нейросетями, понадобятся PyTorch или TensorFlow, а для обработки

естественного языка Transformers и spaCy. По инфраструктуре всё тоже достаточно стандартно. Модели и сервисы упаковываются в Docker, при необходимости масштабируются через Kubernetes, а наружу выставляются через FastAPI или Flask. Адаптивные алгоритмы требуют данных, а значит нужна инфраструктура для сбора логов, их хранения и обработки. Для прототипа это можно упростить, но в продакшене обычно выделяется отдельный сервис.

Для веб-интерфейса на практике выбирают между React, Vue и Angular. У каждого свои сильные стороны, подробно сравню их в разделе 2.3. Отдельно стоит сказать про редактор кода, для алгоритмических задач подойдёт Monaco Editor, тот же движок что стоит в VS Code, или более лёгкий CodeMirror. Для графиков и аналитики есть Recharts и D3, для интерактивных диаграмм React Flow. Если делать стриминг ответов AI в реальном времени понадобится Socket.io-client для работы с WebSocket.

На backend'e будет бизнес-логика, работа с базой данных и интеграция с AI. По языкам основной выбор между Node.js, Python и Go. Node.js удобен, когда фронтенд и бэкенд пишут на одном языке, Go подходит для высоконагруженных систем. Для данного проекта наиболее логичен Python с FastAPI, потому что он хорошо работает с ML-библиотеками, поддерживает асинхронность и WebSocket для стриминга, плюс автоматически генерирует OpenAPI-документацию что удобно для фронтенда. В качестве основной базы данных выбран PostgreSQL, для кэша и сессий в реальном времени Redis, при необходимости хранить большие объёмы логов можно добавить MongoDB.

Для симуляции собеседования критичны две вещи: редактор кода и стриминг ответов. По редактору кода основной выбор между Monaco Editor и CodeMirror. Monaco это тот же движок что в VS Code, мощный, с подсветкой и автодополнением, но тяжёлый. CodeMirror легче и лучше подходит для мобильных версий. Стриминг нужен чтобы AI не выдавал ответ целиком, а печатал его в реальном времени. Технически это можно сделать через WebSocket, Server-Sent Events или long polling. Long polling сейчас почти не

используют. SSE проще в реализации, но работает только в одну сторону, от сервера к клиенту. WebSocket даёт полноценный двусторонний канал и больше гибкости, но сложнее. Если делать совсем реалистично и добавлять видео или аудио, понадобится WebRTC для связи и MediaRecorder API чтобы пользователь мог записать свой ответ и потом переслушать.

Итоговый стек клиентской части выглядит так. Vue с TypeScript, состояние через Zustand или Redux Toolkit, редактор кода на Monaco Editor, стриминг через WebSocket или SSE, графики на Recharts. Бэкенд в рамках работы не разрабатывается, но подразумевается, что он предоставляет API для авторизации, управления сессиями, отправки запросов к AI с поддержкой стриминга и сохранения результатов.

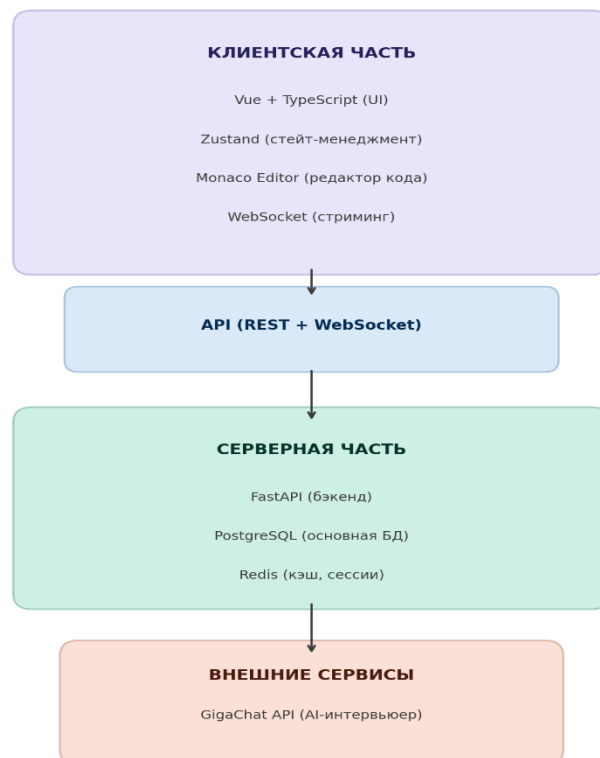


Рисунок 1.2 — Схема технологического стека

На схеме показано из чего состоит система. Верхний слой — это клиент на Vue с TypeScript. Состояние хранится в Zustand, для редактора кода используется Monaco Editor, ответы AI приходят через WebSocket. С

сервером клиент общается по REST для обычных запросов и по WebSocket когда нужен стриминг. Серверная часть работает на FastAPI. В качестве основной базы данных используется PostgreSQL, для кэша и хранения сессий Redis. FastAPI был выбран, потому что хорошо работает с Python-библиотеками для ML, поддерживает асинхронность и автоматически генерирует документацию API. Нижний слой это GigaChat API, он выступает в роли AI-интервьюера. Когда пользователь пишет ответ, сервер отправляет его в GigaChat и стримит полученный ответ обратно клиенту через WebSocket.

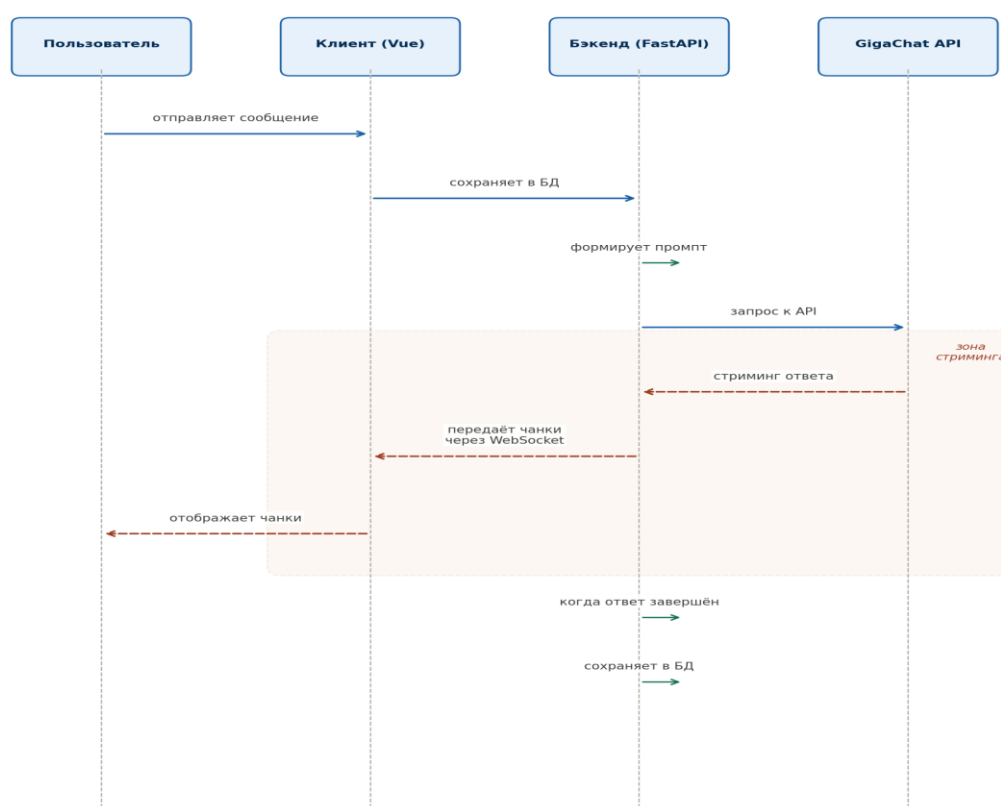


Рисунок 1.3 — Схема взаимодействия компонентов при стриминге

Когда пользователь отправляет сообщение, оно проходит через три звена. Vue-клиент пробрасывает его на бэкенд, бэкенд сохраняет в PostgreSQL и формирует промпт с контекстом диалога, промпт уходит в GigaChat. GigaChat отдаёт ответ не целиком а чанками, бэкенд тут же пробрасывает их клиенту через WebSocket и пользователь видит, как текст

появляется в реальном времени. После завершения генерации бэкенд собирает полный ответ и сохраняет в базу. На схеме зона стриминга выделена отдельно, потому что именно там важна скорость и стабильность соединения.

1.7 Сравнительный анализ frontend-фреймворков и архитектурных подходов.

Поскольку в данной работе разрабатывается именно клиентская часть, от выбора frontend-фреймворка зависит многое. Скорость разработки, доступные библиотеки, производительность приложения, всё это привязано к конкретному инструменту.

Для задачи, где нужен интерактивный тренажер с чатом, редактором кода и стримингом, критерии выбора фреймворка достаточно конкретные. Производительность стоит на первом месте, потому что при стриминге каждую секунду может приходить новый кусок текста и интерфейс должен его отрисовать без задержек. Экосистема тоже важна, нужны готовые библиотеки для редактора кода, графиков, UI-компонентов, чем больше выбор, тем меньше придётся писать самому. Сессия собеседования это сложное состояние: история сообщений, текущий вопрос, таймер, пауза, и фреймворк должен позволять удобно с этим работать. Проект не бесконечный, поэтому кривая обучения тоже имеет значение. Наконец, для сложных моделей данных нужна поддержка TypeScript, типизация сильно снижает количество ошибок.

React от Facebook это, по сути, только библиотека для отрисовки интерфейса. Всё остальное, роутинг, стейт-менеджмент, работу с API, нужно подключать отдельно. Экосистема огромная и виртуальный DOM хорошо справляется с частыми обновлениями, но нужно самому выбирать инструменты и постоянно следить за обновлениями. Vue устроен проще, роутер и стейт-менеджмент уже встроены, документация понятная, экосистема поменьше, но для большинства задач хватает. Angular от Google это

другая крайность, там есть вообще всё из коробки, но порог входа высокий и для прототипа это слишком. Выбран Vue. Основная причина в том что встроенные инструменты позволяют не тратить время на подбор библиотек, а сразу писать код. Monaco Editor и WebSocket подключаются без проблем, для стейт-менеджмента есть Pinia которая заменила Vuex и работает нативно с Composition API. Типизация через TypeScript поддерживается из коробки начиная с Vue 3. С React пришлось бы отдельно выбирать стейт-менеджер и роутер, а Angular слишком тяжёлый, когда работаешь один.

Таблица 1.2 — Сравнение frontend-фреймворков

Критерий	React	Vue	Angular
Производительность	Высокая (Virtual DOM)	Высокая (Virtual DOM)	Высокая (Change Detection)
Экосистема	Огромная	Средняя	Большая
Кривая обучения	Средняя	Низка	Высокая
Встроенный роутер	Нет (React Router)	Да (Vue Router)	Да (RxJS)
TypeScript	Поддержка	Нативная с Vue 3	Нативная
Monaco Editor	Подключается	Подключается	Подключается
WebSocket	Вручную	Вручную	Вручную
Подходит для одного разработчика	Средне	Да	Нет

В итоге выбран Vue 3 с TypeScript. Composition API позволяет собирать сложные компоненты, Monaco Editor для редактора кода подключается без проблем, стейт живёт в Pinia. TypeScript нужен, потому что в приложении много сущностей и без типизации легко запутаться.

Для данного приложения выбран подход SPA, то есть Single Page Application. Вся логика и отрисовка работают на клиенте, а сервер только отдаёт данные через API. Это принципиально важно, потому что во время симуляции собеседования перезагрузка страницы недопустима. SPA позволяет реализовать сложную логику прямо на клиенте, таймеры, стриминг, автосохранение, и создаёт ощущение нативного приложения. Для маршрутизации между страницами используется Vue Router, состояние хранится в Pinia чтобы данные жили в одном месте и все компоненты их видели, для общения с сервером подключён axios. В перспективе SPA можно упаковать в PWA, тогда пользователь сможет установить приложение на

телефон и просматривать историю без интернета. Для ВКР это опционально, но в планах развития стоит учесть.

На диаграмме видно, что React набирает больше всего баллов за счёт экосистемы, но Vue выигрывает в простоте обучения. Angular проигрывает именно по этому критерию, порог входа там заметно выше. Для данного проекта выбран Vue, потому что разница в экосистеме не критична, Monaco Editor и WebSocket подключаются без проблем, а выигрыш в скорости освоения при работе в одиночку важнее.

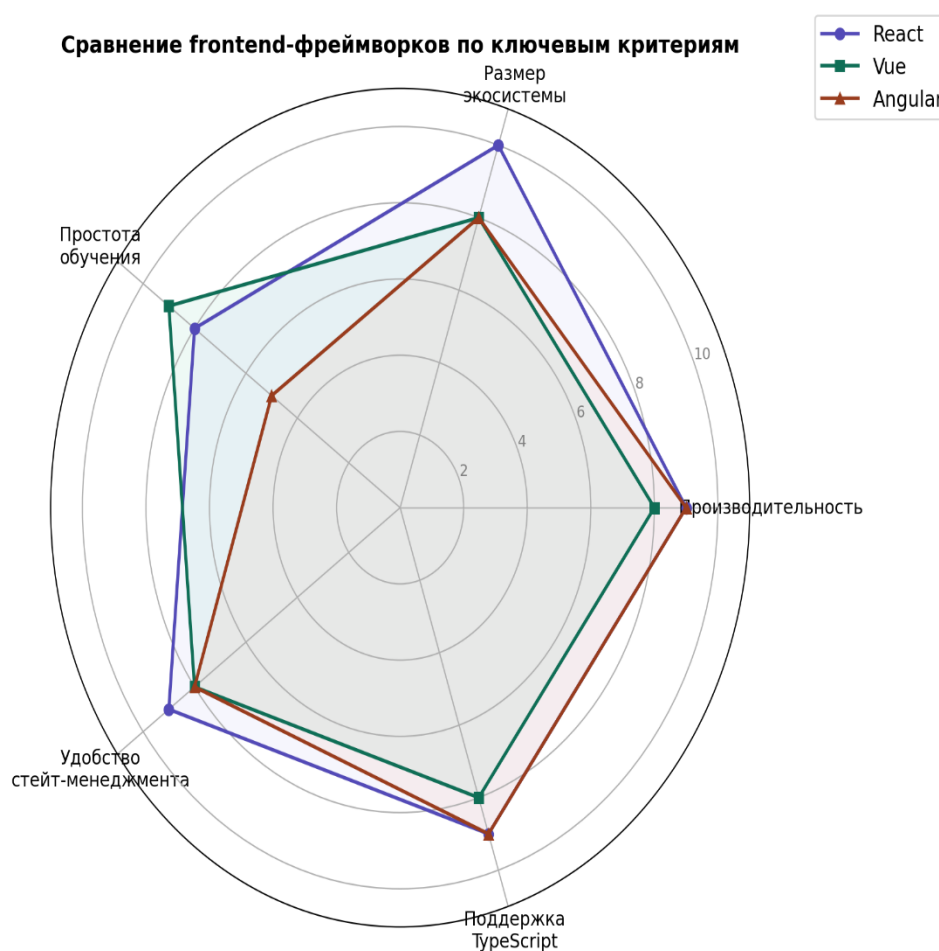


Рисунок 1.4 — Сравнение frontend-фреймворков по ключевым критериям

1.8 Алгоритм выбора методов и технологий

Чтобы не выбирать технологии наобум, я выстроил алгоритм, по которому принимал решения. Он пригодится и для других проектов.

Сначала нужно понять с чем имеем дело. Данные в системе бывают двух видов. Структурированные это профиль пользователя, история ответов, метки задач, их можно хранить в реляционной БД. Неструктурированные это текстовые ответы на открытые вопросы, для работы с ними нужны AI-модели. Система должна не просто выдавать вопросы, а вести полноценный диалог, анализировать ответы даже когда они не сводятся к да или нет, и давать обратную связь в реальном времени. Ответ должен приходиться за 2–5 секунд, иначе диалог рвётся и пропадает ощущение живого общения. При этом качество диалога должно быть таким чтобы он был похож на разговор с реальным интервьюером, шаблонные ответы не подойдут. Ресурсов обучать свою модель с нуля нет, значит нужно брать готовую.

На втором шаге нужно понять какой метод подходит под требования. Курирование контента в стиле LeetCode отпадает сразу, потому что оно не генерирует диалог и не анализирует ответы, только выдаёт готовые задачи. Классические ML-алгоритмы тоже не подходят, они работают со структурированными данными, но не понимают смысл текста. Правила можно написать для простых случаев, но для полноценного диалога их потребуется слишком много и всё равно все варианты не покроешь. Остаются LLM, большие языковые модели. Это единственный инструмент, который умеет и генерировать диалог и анализировать текст и делать это в реальном времени. Вопрос в том какую конкретно модель выбрать.

На рынке есть open-source модели типа Llama и Mistral, их можно поднять у себя, но нужны сервера и с русским языком у них плохо. Есть проприетарные, GPT, Claude, GigaChat, они работают через API. Выбор пал на GigaChat, потому что он обучался на русских текстах, а собеседования в проекте идут на русском. API поддерживает стриминг, есть бесплатный лимит для тестирования, интеграция через обычный HTTP. GPT рассматривался, но он хуже работает с русской IT-терминологией и после 2022 года доступ к нему из России усложнился.

GigaChat это мозг системы, он генерирует вопросы и оценивает ответы. Но он не хранит историю, не управляет пользователями и не формирует отчёты, для этого нужен свой бэкенд. Работает это так: пользователь пишет сообщение через интерфейс на Vue, бэкенд на FastAPI получает его и добавляет в историю сессии. Далее бэкенд собирает промпт с учётом контекста диалога и отправляет в GigaChat API. Ответ приходит частями через стриминг, бэкенд тут же пробрасывает их клиенту по WebSocket и пользователь видит эффект печати. Когда ответ закончен бэкенд сохраняет его в PostgreSQL. Такой гибридный подход позволяет получить качество диалога от GigaChat, при этом все данные хранятся на своём бэкенде и можно строить любую аналитику.

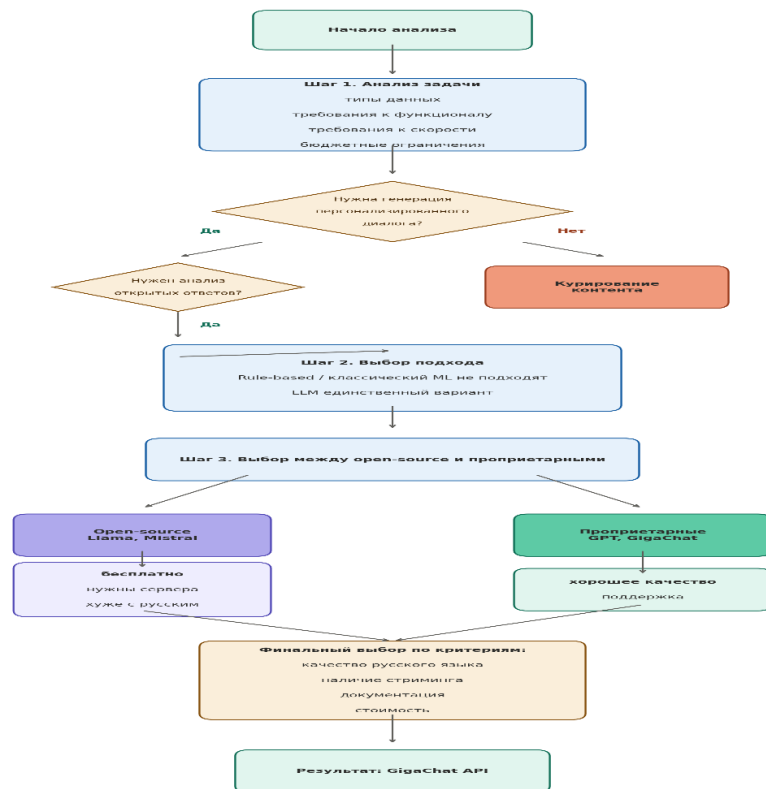


Рисунок 1.5 — Блок-схема алгоритма выбора методов и технологий

На рисунке 1.5 показано как происходил выбор методов и технологий. Сначала анализируются требования задачи, и уже на этом этапе отсекается курирование контента и классический ML, потому что они не умеют

генерировать диалог и анализировать открытые ответы. Остаются только LLM. Дальше выбор между open-source и проприетарными моделями, и по совокупности критериев, русский язык, стриминг, документация, стоимость, выбран GigaChat.

1.9 Выводы по теоретико-аналитической части

Во второй главе были рассмотрены три метода подготовки к собеседованиям. Курирование контента — это самый простой подход, но он статичный и не подстраивается под пользователя. Адаптивные системы решают эту проблему, только реализовать их сложно. Симуляция ближе всего к реальному собеседованию, зато с оценкой качества там пока плохо. Ни один метод в одиночку не справляется, нужна комбинация. Интерфейс строится на Vue с TypeScript. Бэкенд на FastAPI, редактор кода на Monaco Editor. Стриминг ответов через WebSocket. В качестве AI-модели взят GigaChat за русский язык и поддержку стриминга. Вся система работает по гибриднему принципу, бэкенд отвечает за данные и сессии, GigaChat за контент. Проектирование и реализация описаны в следующих главах.

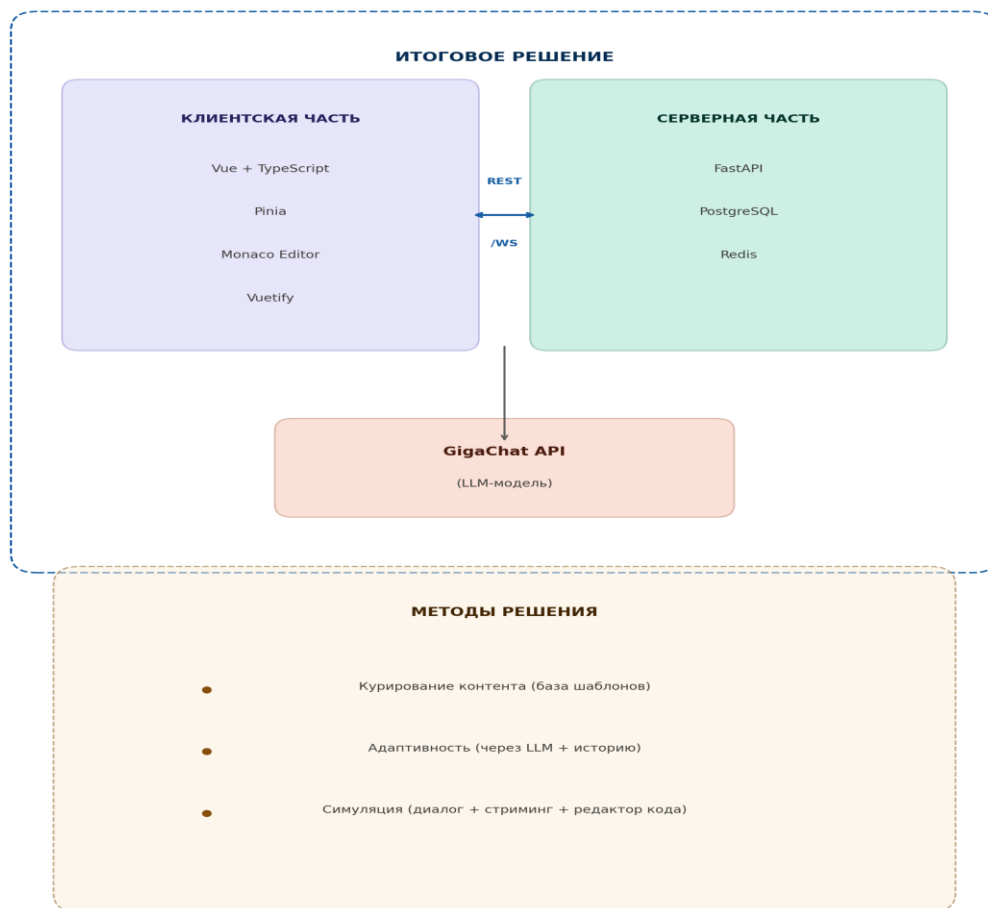


Рисунок 1.6 — Итоговая схема выбранного решения

На рисунке 1.6 показана итоговая архитектура решения. Клиентская часть на Vue с TypeScript общается с серверной частью на FastAPI через REST и WebSocket. Сервер хранит данные в PostgreSQL, кэш в Redis, и обращается к GigaChat API для генерации диалога. В нижней части схемы показано какие методы реализуются через эту архитектуру: курирование контента через базу шаблонов, адаптивность через LLM и историю пользователя, симуляция через диалог со стримингом и редактором кода.

2. ПРАКТИЧЕСКАЯ ЧАСТЬ СТАРТАП-ПРОЕКТА VIEWTRAIN

2.1 Описание стартап-проекта ViewTrain

2.1.1 Описание проекта и продукта

ViewTrain это платформа для подготовки к техническим собеседованиям в IT. Идея появилась из личного опыта, большинство кандидатов готовятся хаотично, собирая инструменты по частям. Кто-то сидит на LeetCode, кто-то ищет партнёра для mock-интервью, кто-то смотрит видеоразборы. Целостного пути нет. ViewTrain закрывает эту проблему, пользователь проходит тренировочное собеседование с AI который адаптируется под его уровень и даёт обратную связь.

Продукт состоит из веб-приложения с тремя основными модулями. Чат с AI-интервьюером на базе GigaChat, редактор кода для алгоритмических задач на Monaco Editor, и модуль аналитики с детальным разбором ответов после завершения сессии.

Целевая аудитория — это разработчики, которые готовятся к собеседованиям. В основном джуниоры и мидлы, у них мало опыта прохождения интервью и много стресса. Конкуренция на рынке высокая, на одну вакансию приходит десятки откликов. Платформ, заточенных именно под тренировку собеседований в России почти нет.

Конкуренты разобраны в первой главе с технической стороны. С точки зрения бизнеса картина такая. LeetCode Premium стоит 35 долларов в месяц, аудитория несколько миллионов пользователей, но платформа заточена только под алгоритмы. Pramp бесплатен, монетизируется через партнёрства с работодателями, но качество сессий нестабильно. AlgoExpert продаёт годовой доступ за 99 долларов, контент высокого качества, но формат пассивный.

InterviewBit перешёл в модель Scaler Academy с ценником от 3000 долларов за курс.

Прямых конкурентов в России с AI-интервьюером нет. Яндекс.Практикум и Хекслет обучают программированию, но не тренируют прохождение собеседований. Ниша свободна.

Бизнес-модель freemium. Бесплатно можно пройти две сессии в неделю с базовым разбором ответов. Подписка снимает ограничения и открывает расширенную аналитику, персональные рекомендации и доступ ко всем направлениям. Цена планируется в районе 500–900 рублей в месяц, это дешевле репетитора и сопоставимо с подпиской на LeetCode Premium. Годовая подписка со скидкой.

Выбор freemium обоснован спецификой аудитории. Значительная часть пользователей — это студенты и начинающие специалисты, для которых платный вход может стать барьером. Бесплатные сессии снижают порог входа. Пользователи, которые готовятся системно, быстро упираются в лимит и переходят на подписку.

Масштабирование в несколько направлений. Первое это новые специализации, сейчас Backend и Frontend, дальше Data Science, DevOps, QA. Каждое новое направление расширяет аудиторию без принципиальных изменений в архитектуре.

Второе это корпоративный сегмент. HR-отделы компаний могут использовать платформу для предварительной оценки кандидатов или рекомендовать её перед интервью. Корпоративные лицензии — это другой канал монетизации с более высоким средним чеком.

Третье это партнёрства с учебными заведениями, платформа подходит как инструмент для курсов по подготовке к трудоустройству. Вузы заинтересованы в инструментах, которые повышают показатели трудоустройства выпускников.

Четвёртое это мобильное приложение (PWA). Прохождение сессии удобнее на десктопе, но просмотр результатов и рекомендаций можно делать с телефона.

Пятое это локализация. Формат технического собеседования универсален, перевод на английский язык открывает доступ к международному рынку.

2.2 Реализация клиентской части

2.2.1 Этапы разработки

Разработка клиентской части велась в несколько этапов. Первым делом был собран каркас на Vue 3 с TypeScript. Для сборки взят Vite, он заметно быстрее Webpack при горячей перезагрузке и это экономит время, когда интерфейс постоянно меняется. Навигация через Vue Router, состояние в Pinia, UI-компоненты из Vuetify. Сразу после каркаса была сделана аутентификация через JWT-токены с guard на маршрутах. Основная часть работы ушла на модуль собеседования, там чат с AI, редактор кода, таймер и панель прогресса, каждый компонент тестировался изолированно. Модуль аналитики делался в последнюю очередь, потому что он зависит от данных, которые накапливаются в процессе собеседования.

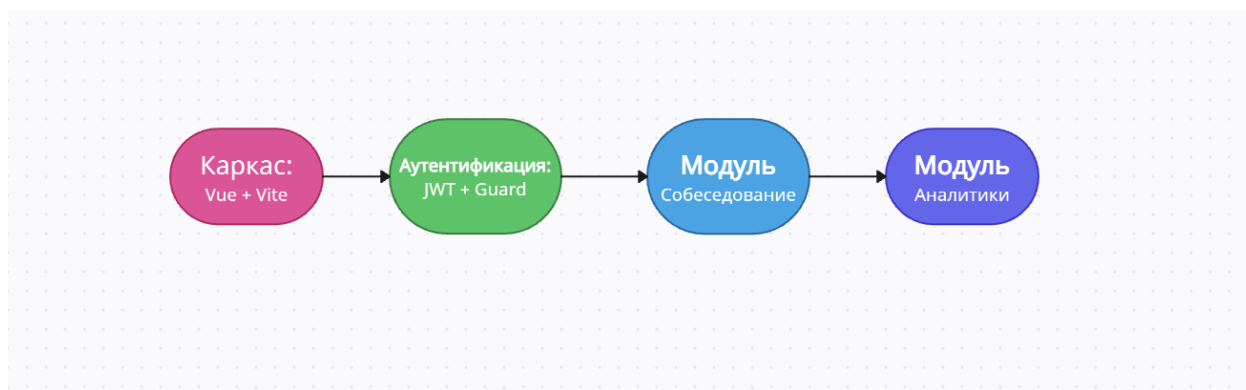


Рисунок 2.1 — Этапы разработки клиентской части

2.2.2 Компонентная архитектура на Vue

Приложение разбито на компоненты по принципу единственной ответственности. Каждый компонент отвечает за одну вещь и не лезет в чужую логику. На верхнем уровне стоит `App.vue` который содержит роутер и общий `layout`. Внутри `layout` лежат страницы: `LoginPage`, `DashboardPage`, `InterviewPage`, `AnalyticsPage`. Каждая страница — это композиция из более мелких компонентов. Сложнее всего устроена `InterviewPage`, там три панели работают одновременно, и все подписаны на один `store` в `Pinia`. Чат занимает центральную часть экрана. Рядом с ним панель прогресса и панель ввода, которая меняет форму в зависимости от типа вопроса. Повторяющаяся логика вынесена в `composables` через `Composition API`.

Проект организован по стандартной для Vue 3 структуре. В корне лежит `src`, внутри него папки `components`, `pages`, `stores`, `composables`, `api` и `assets`. В `components` лежат переиспользуемые компоненты, `ChatMessage`, `CodeEditor`, `ProgressBar`, `TimerWidget`. Каждый в своей папке вместе со стилями и тестами. В `pages` четыре страницы, `LoginPage`, `DashboardPage`, `InterviewPage`, `AnalyticsPage`. `Stores` это `Pinia` хранилища, их три: `useAuthStore` для авторизации, `useSessionStore` для текущей сессии и `useHistoryStore` для истории собеседований. `Composables` уже описывались выше, там `useWebSocket`, `useTimer`, `useCodeRunner`. В папке `api` лежат функции для обращения к бэкенду, обернутые в `axios` с автоматической подстановкой JWT-токена. `Assets` содержит иконки и глобальные стили.

2.2.3 Модуль диалога с AI-интервьюером

Модуль диалога — это центральная часть приложения. Выглядит как обычный мессенджер, сообщения пользователя справа, AI слева. Когда пользователь отправляет ответ, он сразу появляется в чате и AI начинает печатать. Ответ приходит кусками через `WebSocket` и дописывается

посимвольно. Если соединение рвётся useWebSocket пробует переподключиться, не получилось показывает кнопку повторить.

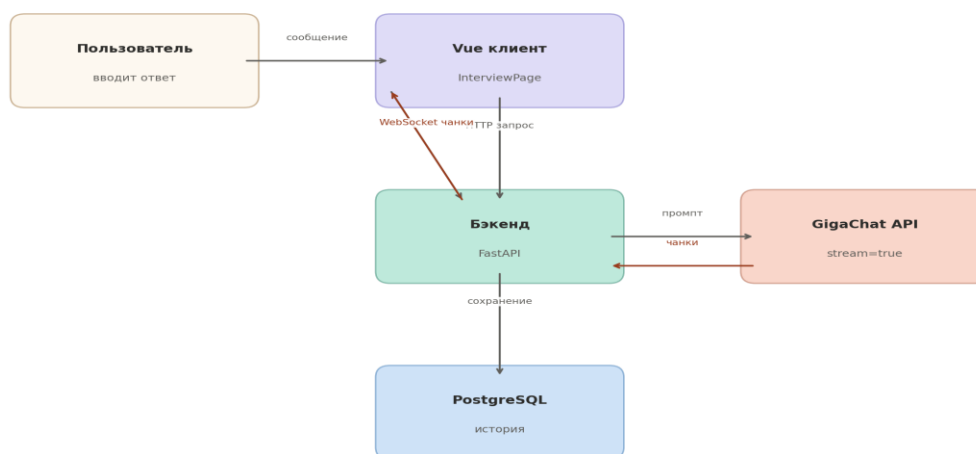


Рисунок 3.2 — Поток данных в модуле диалога

Ответы от GigaChat приходят в plain text, но часто содержат markdown разметку. Заголовки, списки, блоки кода с указанием языка. Чтобы всё это нормально отображалось в чате, подключена библиотека marked которая парсит markdown в HTML. Для блоков кода дополнительно используется highlight.js, он подсвечивает синтаксис прямо в сообщении. На практике это важно, потому что AI часто показывает примеры кода в своих объяснениях и без подсветки читать их тяжело. Отдельно пришлось обработать ситуацию, когда блок кода приходит частями через стриминг. Если просто парсить каждый чанк отдельно, markdown ломается, потому что открывающие и закрывающие тройные кавычки могут оказаться в разных чанках. Решается буферизацией, пока блок кода не закрыт он рендерится как обычный текст, а после закрытия перерисовывается с подсветкой.

2.2.4 Редактор кода и панель ввода

Панель ввода меняется в зависимости от того какой вопрос задаёт AI. Если вопрос поведенческий, появляется обычное текстовое поле. Если

алгоритмический, вместо текстового поля загружается Monaco Editor с подсветкой синтаксиса и автодополнением. Пользователь выбирает язык, пишет код и запускает его прямо в интерфейсе, результат появляется под редактором. Monaco тяжёлый сам по себе, поэтому грузится через lazy loading, иначе первая загрузка страницы была бы слишком долгой. Ещё есть режим системного дизайна, там работает Vue Flow и пользователь строит архитектуру из блоков и стрелок, результат уходит на бэкенд как JSON.

2.2.5 Модуль аналитики и визуализации прогресса

После завершения сессии пользователь попадает на экран аналитики. Там показывается сводная оценка по категориям, алгоритмы, теория, коммуникация. Оценки приходят от GigaChat в последнем запросе, бэкенд парсит их и сохраняет в базу. На клиенте оценки отрисовываются через Recharts в виде радарной диаграммы. Кроме оценок есть текстовая обратная связь, AI пишет, что было хорошо и над чем стоит поработать. Этот текст приходит обычным сообщением и рендерится с поддержкой markdown. Если у пользователя есть история из нескольких сессий, на дашборде появляется график динамики. Видно, как менялись оценки от сессии к сессии и в каких темах есть прогресс. Данные для графика берутся из Pinia store который подтягивает историю с бэкенда при загрузке дашборда.

После авторизации пользователь попадает на дашборд. Там видно сколько сессий пройдено, средняя оценка и список последних собеседований. Каждая сессия в списке показывает дату, тип собеседования, уровень сложности и итоговую оценку. Можно нажать на любую и провалиться в детальный отчёт. Данные для дашборда лежат в useHistoryStore, запрос к бэкенду делается один раз. У новых пользователей вместо списка кнопка начать первое собеседование.

2.2.6 Интеграция с GigaChat API

Клиент напрямую к GigaChat не обращается, всё идёт через бэкенд. Бэкенд берёт шаблон промпта, подставляет туда историю диалога и текущий ответ пользователя, и отправляет запрос с параметром `stream=true`. GigaChat начинает отдавать токены по одному, бэкенд их принимает и пробрасывает клиенту через `WebSocket`, на клиенте `useWebSocket` слушает чанки и дописывает их в текущее сообщение AI. Для каждого типа вопроса есть свой шаблон промпта куда подставляется уровень сложности и специализация пользователя, так что GigaChat задаёт вопросы с учётом того что уже обсуждалось.

Промпты для GigaChat не пишутся каждый раз с нуля, они собираются из шаблонов. Для алгоритмического вопроса один шаблон, для поведенческого другой, для системного дизайна третий. В каждый подставляются переменные: уровень пользователя, специализация, номер вопроса и краткое содержание предыдущих ответов. Без контекста предыдущих ответов GigaChat иногда закикливался на одной теме, после добавления истории в промпт это прекратилось. Шаблоны хранятся на бэкенде в базе данных, администратор может их редактировать через отдельный интерфейс не трогая код.

GigaChat API иногда отвечает ошибками. Бывает 429 когда превышен лимит запросов, бывает 500 когда на стороне API что-то упало, бывает таймаут, когда ответ не пришёл за 30 секунд. 429 обрабатывается повтором через секунду, 500 тоже повтором, но с нарастающей задержкой. Таймаут сразу уходит клиенту как ошибка. Пользователь видит не код ошибки, а нормальный текст и кнопку повторить, то, что он набрал в поле ввода не пропадает.

2.2.7 Управление состоянием через Pinia

Store в приложении три штуки. `useAuthStore` самый простой, в нём токен и данные пользователя. При логине токен пишется в `localStorage` и подставляется в `axios`. `useSessionStore` устроен сложнее, в нём хранится всё что

связано с текущим собеседованием. Сообщения, вопрос, таймер, какая панель ввода сейчас активна. Когда приходит чанк от WebSocket он дописывается сюда и Vue сам перерисовывает нужные компоненты. useHistoryStore подтягивает список прошлых сессий для дашборда. Pinia выбрана вместо Vuex потому что API проще, TypeScript поддерживается нативно и не нужны мутации. Когда работаешь один это экономит время.

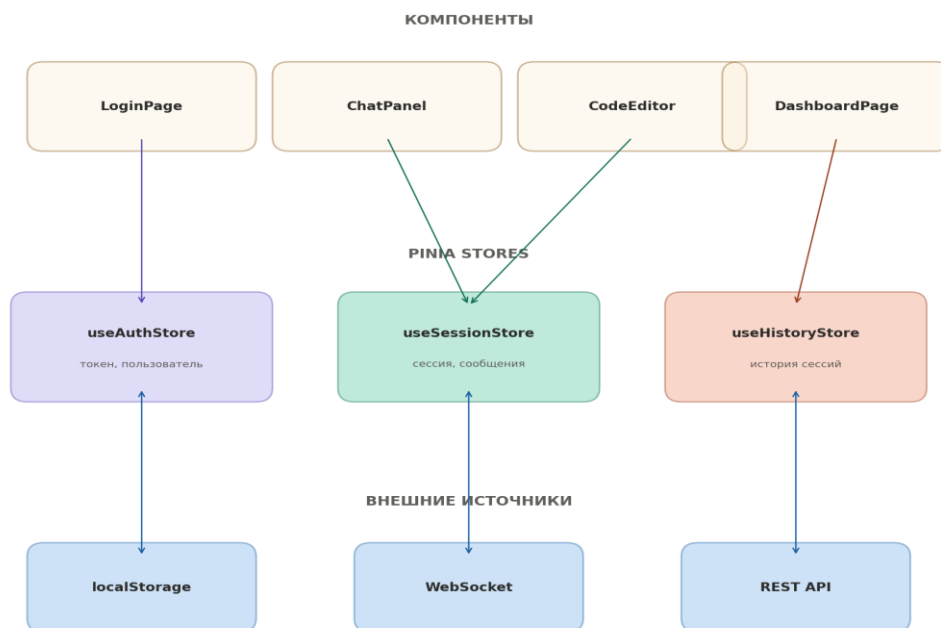


Рисунок 3.3 — Связь Pinia stores с компонентами

2.2.8 Маршрутизация и защита маршрутов

Маршрутизация сделана через Vue Router. Маршрутов немного: `/login`, `/dashboard`, `/interview/:id` и `/analytics/:id`. Все кроме `/login` защищены `navigation guard` который проверяет есть ли токен в `useAuthStore`. Нет токена, редирект на логин. Отдельно обрабатывается ситуация когда токен есть но просрочен. При первом запросе к бэкенду приходит 401, `axios interceptor` ловит его, чистит store

и кидает на страницу входа. Переходы между страницами происходят без перезагрузки, это SPA. Во время активной сессии собеседования уход со страницы блокируется через `beforeRouteLeave`, пользователь видит предупреждение что сессия не завершена.

2.2.9 Тестирование и валидация

Для тестов использовался Vitest. Писались unit-тесты на компоненты, проверял что сообщения нормально попадают в store и что таймер не сбивается. `useWebSocket` тестировался отдельно на обрыв соединения. Полный сценарий прогонял вручную, стриминг проверял на медленном интернете. TPI в пределах 3 секунд, токен от AI за 2.

2.2.10 Перспективы развития

Текущая версия — это рабочий прототип, дальше есть куда расти. Первое что напрашивается это PWA, чтобы приложение можно было поставить на телефон и просматривать историю без интернета. Второе это голосовой режим, на реальном собеседовании говоришь, а не печатаешь, и было бы полезно тренировать именно устную речь через WebRTC и распознавание голоса. Третье это расширение аналитики, сейчас отчёт достаточно простой, а можно добавить сравнение с другими пользователями и рекомендации что повторить. Наконец, сейчас промпты защиты в шаблоны, а в будущем можно дать администраторам интерфейс для их редактирования без участия разработчика.

2.2.11 Принципы UX-дизайна для стрессовых сценариев

Собеседование — это стресс, и интерфейс, который его имитирует не должен добавлять стресса сверху. Главное правило: в каждый момент времени пользователь видит только то что ему нужно прямо сейчас. Ничего лишнего на экране. У каждого состояния системы есть визуальный индикатор, AI думает,

ответ записан, время заканчивается. Кнопка отправки и область ввода выделены контрастно чтобы глаз цеплялся за них. Пауза, редактирование ответа до отправки, запрос подсказки, всё это доступно чтобы пользователь чувствовал, что он управляет процессом, а не наоборот.

2.2.12 Информационная архитектура и навигация

Навигация строится вокруг одного сценария, пользователь пришёл чтобы пройти собеседование. После логина он попадает на дашборд, где видит историю и кнопку начать новую сессию. При нажатии открывается выбор параметров: специализация, уровень сложности, тип интервью. После этого пользователь попадает на основной экран собеседования и остаётся там до конца сессии. Переход на другие страницы во время сессии заблокирован, это сделано намеренно чтобы имитировать условия реального интервью, где нельзя просто встать и уйти. После завершения открывается экран аналитики с отчётом. Оттуда можно вернуться на дашборд. Всего четыре экрана, ничего лишнего.

2.2.13 Описание ключевых экранов

Основной экран собеседования разделён на три области. Левая панель показывает структуру интервью, какие темы будут, сколько вопросов в каждой, какие уже пройдены. Это помогает пользователю ориентироваться, где он сейчас находится в процессе. Центральная область — это чат. Там история диалога, сообщения AI сверху, ответы пользователя снизу. Внизу чата поле ввода, которое меняется в зависимости от вопроса. Правая панель появляется только когда нужна, например при алгоритмическом вопросе там открывается редактор кода, при системном дизайне конструктор диаграмм. Если вопрос поведенческий правая панель скрывается и чат занимает больше места. Сверху над всем этим таймер и кнопки управления сессией, пауза и завершить.

2.2.14 Выбор UI-библиотеки и компонентов

В качестве UI-библиотеки взят Vuetify. Там готовые компоненты, кнопки, поля ввода, диалоговые окна, карточки, всё уже протестировано на доступность. Система тем позволяет поменять цвета и типографику не переписывая компоненты. Рассматривался Quasar но у Vuetify документация лучше. Нестандартные вещи вроде редактора кода и конструктора диаграмм Vuetify не покрывает, для них подключены Monaco Editor и Vue Flow. Визуально они подогнаны под общий вид через CSS-переменные.

2.2.15 Модель данных клиентской части

На клиенте три основные сущности. Пользователь это id и имя, больше ничего на фронте не нужно. Сессия собеседования содержит id, id пользователя, статус (активна, на паузе, завершена), тип интервью, уровень сложности и специализацию. История сообщений — это массив объектов, у каждого id, отправитель (user или ai), тип сообщения (текст, код, диаграмма), содержимое и метка времени. Отдельно хранится текущий ответ пользователя, он живёт в компоненте панели ввода и попадает в store только после отправки. Это сделано чтобы каждое нажатие клавиши не вызывало перерисовку всего чата. Все типы описаны через TypeScript интерфейсы, если где-то передаётся не тот тип компилятор ругается сразу, а не в рантайме.

2.2.16 Реактивность и управление состоянием

Реактивность Vue позволяет не думать о том, когда перерисовывать компоненты. Данные меняются в store и всё что на них подписано обновляется само. Приходит чанк от WebSocket, он дописывается в массив сообщений, ChatPanel дорисовывает текст, ProgressBar пересчитывает прогресс. Единственное место, где пришлось вмешаться это скролл чата. По умолчанию при добавлении текста чат не прокручивается вниз. Написал watcher который

следит за длиной последнего сообщения и дёргает scrollTo. Мелочь, но без неё пользователь не видит новый текст пока сам не проскроллит.

2.2.17 Адаптивность и доступность интерфейса

Интерфейс рассчитан на экраны от 1200px до 1920px и выше. На широких мониторах все три панели видны одновременно. На экранах поменьше правая панель сворачивается и открывается по кнопке, чтобы чату хватало места. Отдельно гонял ахе DevTools, клавиатурная навигация работает, aria-label на кнопках стоят, контраст в норме.

2.2.18 Экран авторизации и регистрации

При открытии сайта пользователь попадает на лендинг. Справа форма входа, слева промо-блок с описанием платформы. Вход по email и OTP-коду, четыре цифры приходят на почту. Новые пользователи заполняют отдельную форму с именем, телефоном и паролем. После этого открывается приветственный экран и можно переходить к выбору направлений. Промо-блок слева не меняется на протяжении всех шагов авторизации.

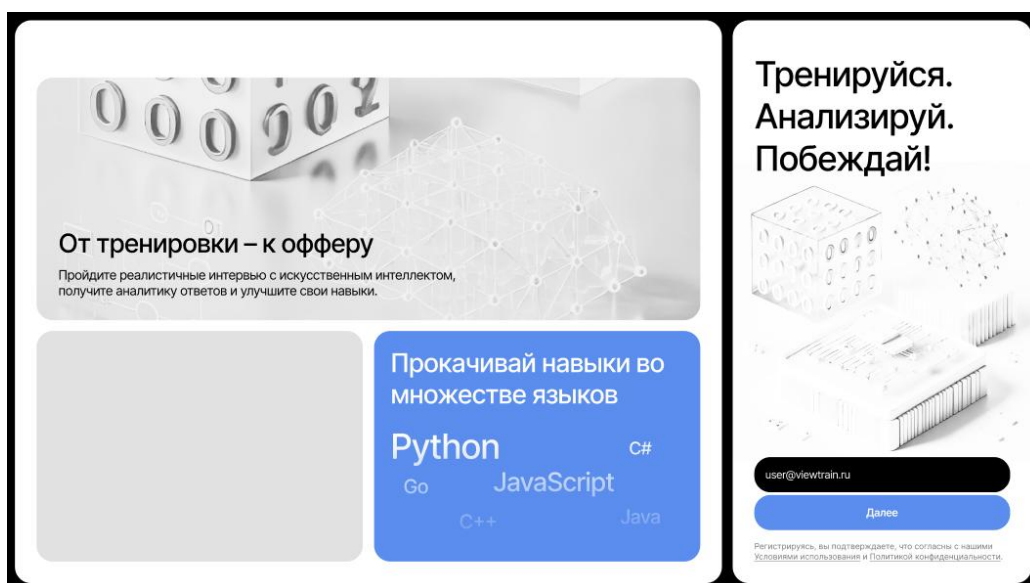


Рисунок 2.1 — Экран авторизации

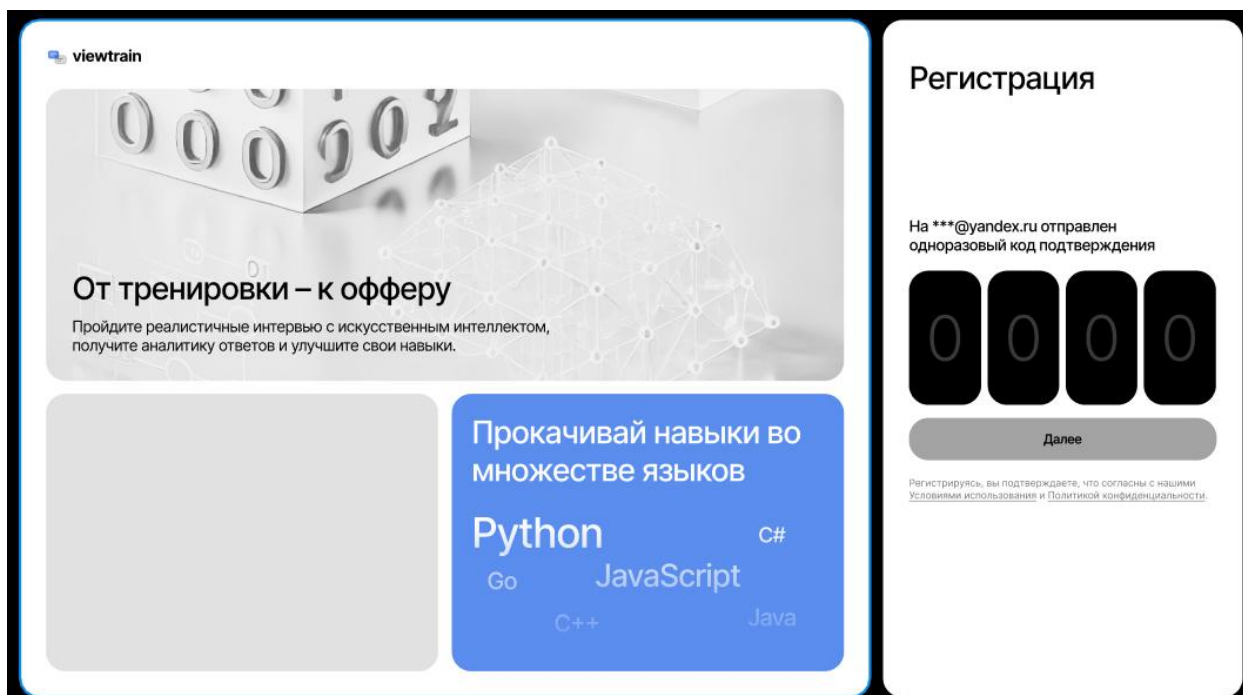


Рисунок 2.2 — Ввод OTP-кода подтверждения

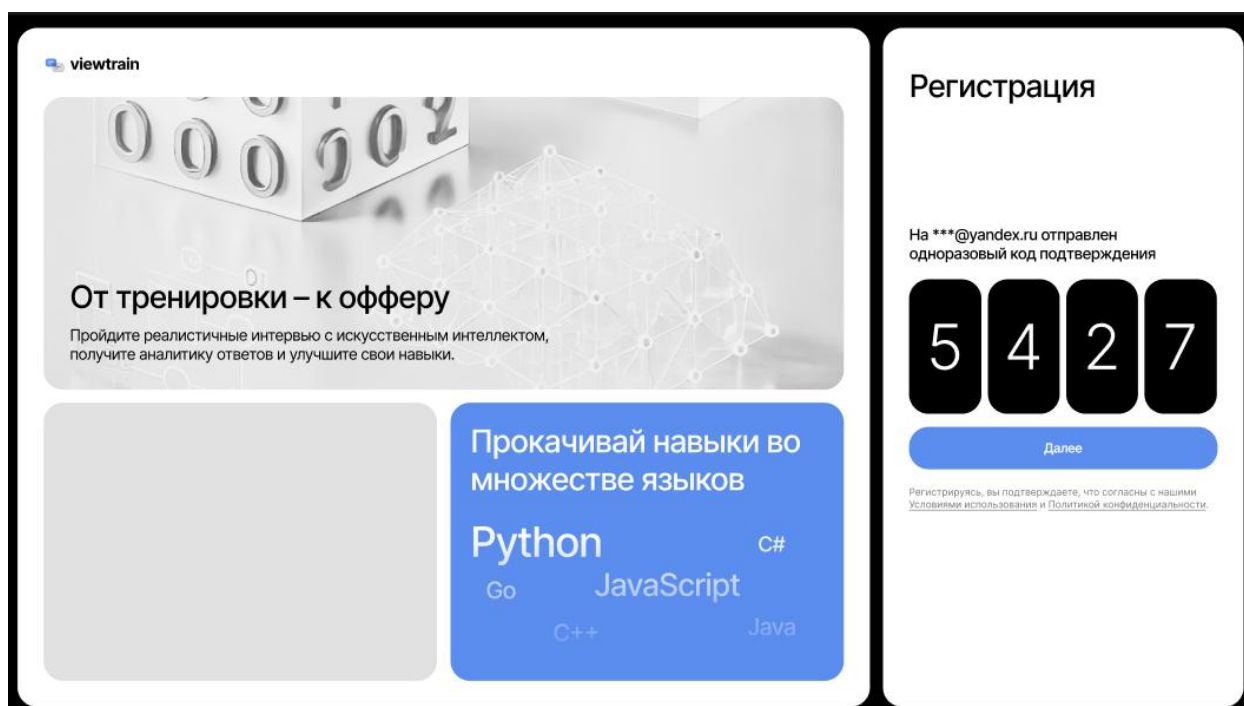


Рисунок 2.3 — Ввод OTP-кода (заполненный)

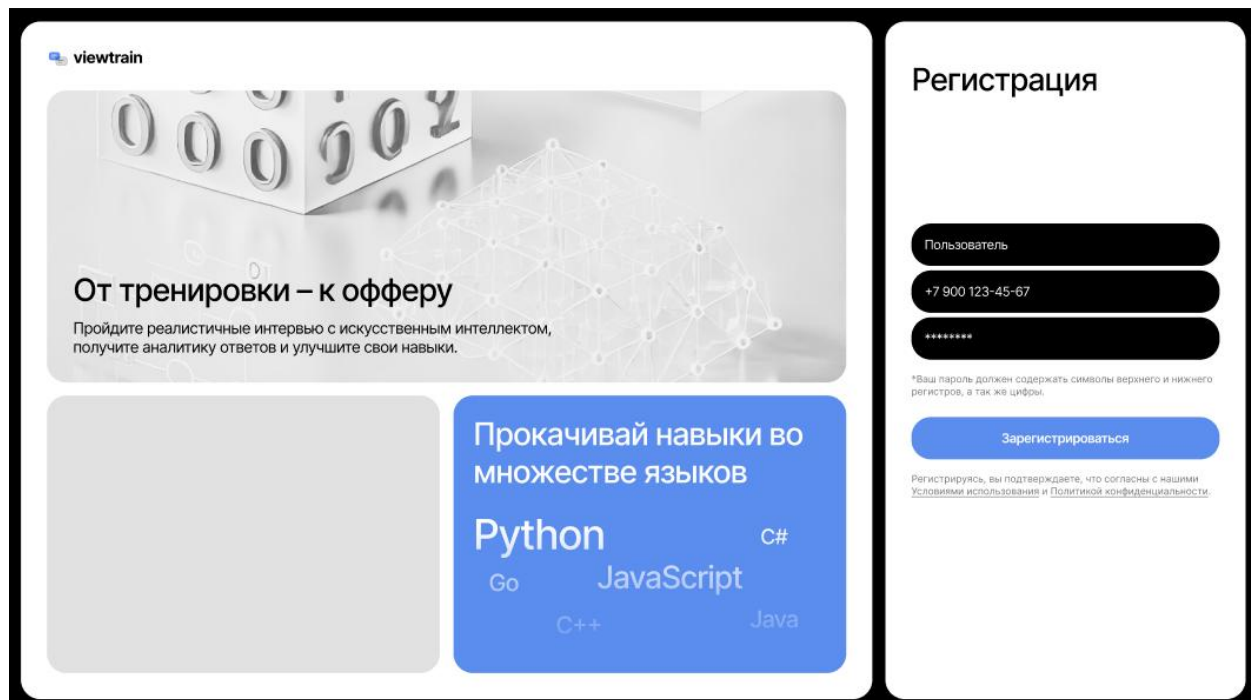


Рисунок 2.4 — Форма регистрации (имя, телефон, пароль)

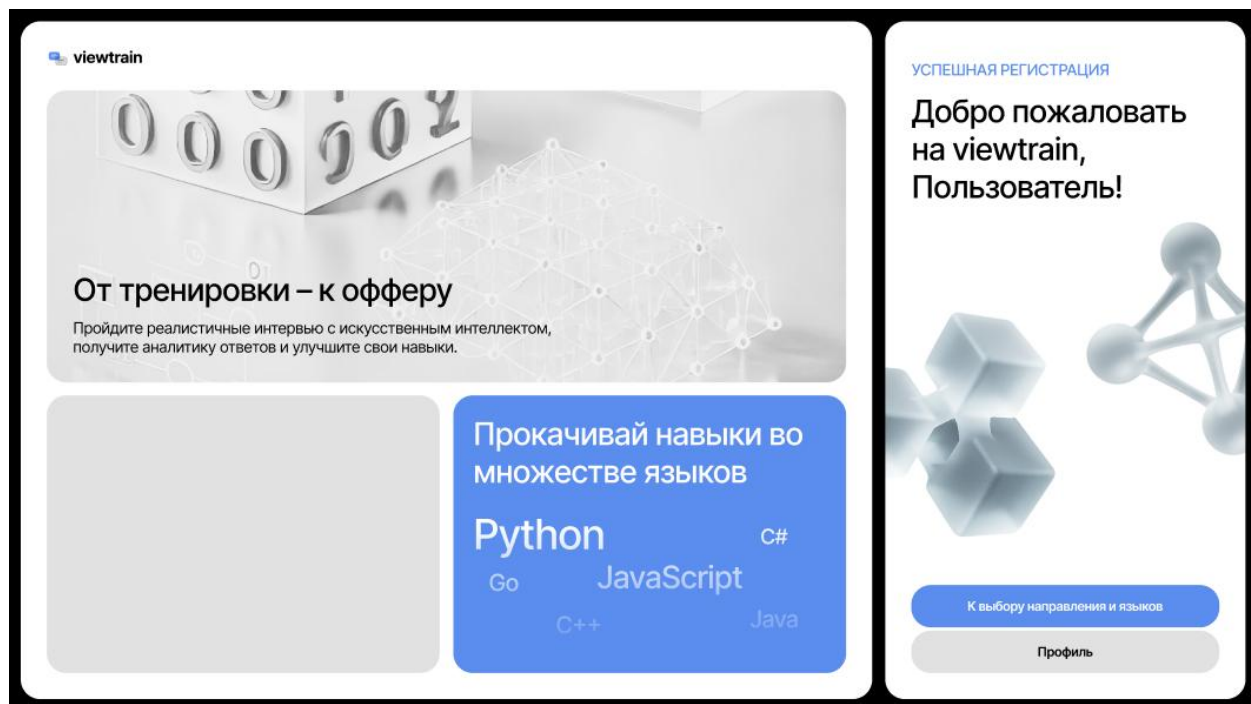


Рисунок 2.5 — Успешная регистрация

2.2.19 Экран результатов сессии

Когда сессия заканчивается пользователь видит разбор. Слева оценки по категориям, технические знания 92%, коммуникация 74%,

структурированность 88% и общий балл 85. Справа список вопросов, каждый с пометкой отлично, хорошо или можно лучше. Любой вопрос раскрывается. Внутри свой ответ и что сказал AI. По Two Sum , например AI написал, что структура данных верная, но лучше начинать с наивного решения, а потом оптимизировать.

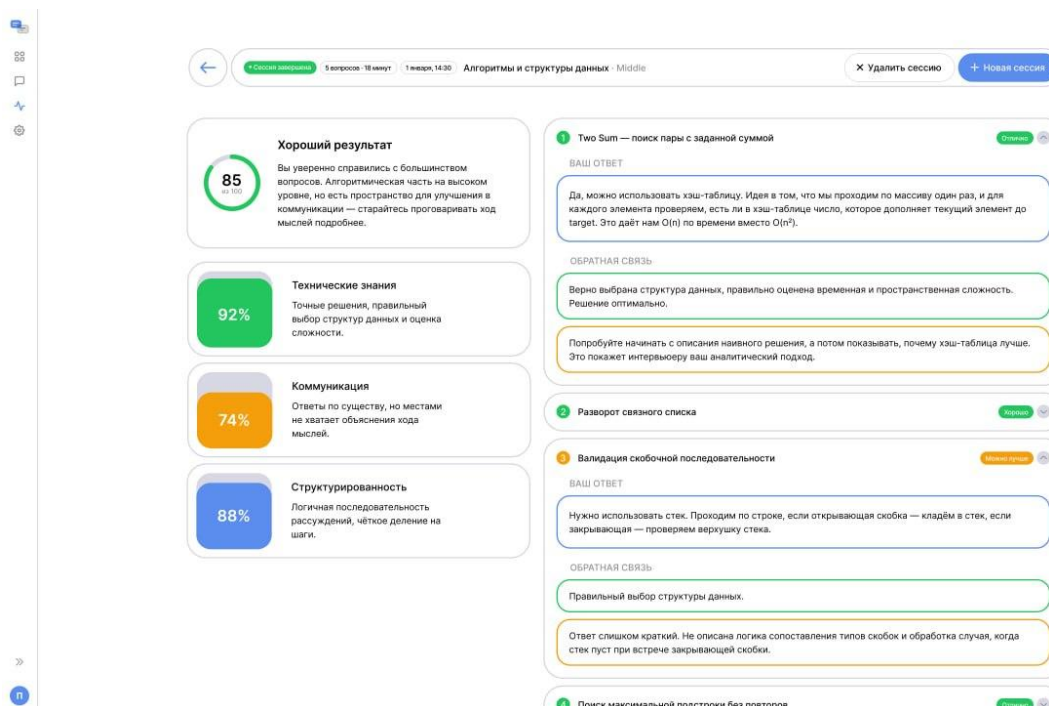


Рисунок 2.6 — Экран с итогом сессии

2.2.20 Дашборд и статистика

После авторизации пользователь попадает на дашборд. Сверху полоса активности по дням недели, видно сколько дней подряд тренируешься и сколько осталось до рекорда. Ниже четыре карточки с цифрами: всего сессий, средняя оценка, лучший результат и серия дней подряд. График динамики оценок стоит справа, там есть фильтры по категориям. Внизу история сессий и три блока с рекомендациями, слабое место, следующий шаг, мотивация. Если плохо шли графы система предложит потренировать именно их.

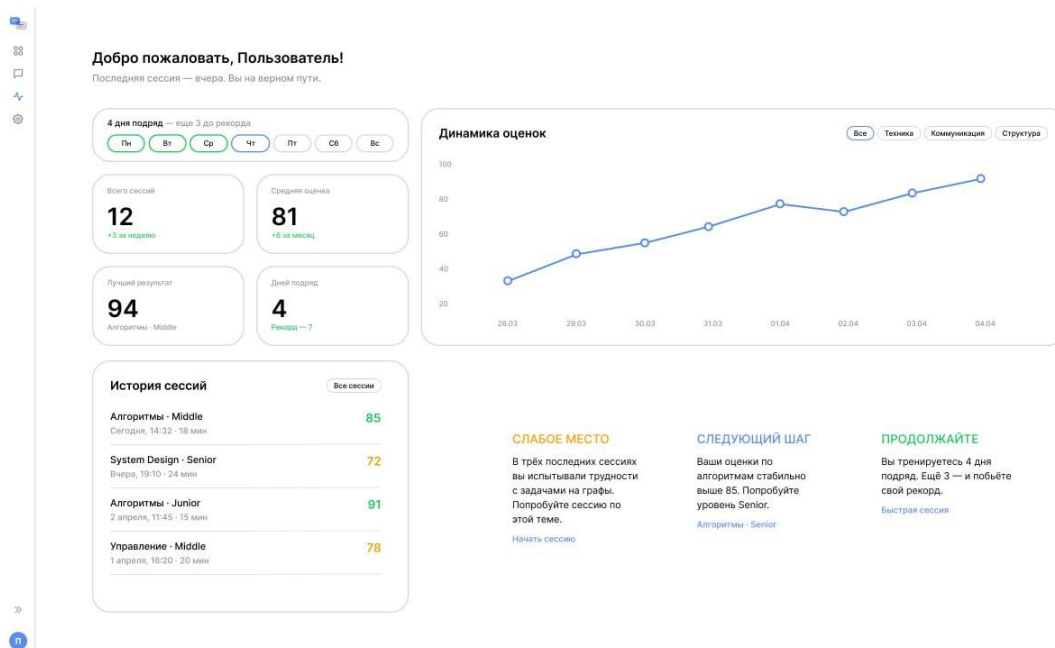


Рисунок 2.7 — Настройки профиля и статистика

2.3. Экономическое обоснование

2.3.1 Затраты на разработку

Затраты на разработку складываются из нескольких статей. Клиентская часть заняла 4 месяца, один разработчик. По рыночной ставке фронтендера в Москве это примерно 320–400 тысяч рублей. Бэкенд и интеграция с GigaChat стоят сопоставимо, итого на разработку MVP порядка 650–800 тысяч рублей. Дизайн интерфейса и макеты выполнялись параллельно с разработкой, отдельной статьёй не выделяются.

2.3.2 Операционные расходы

Серверная инфраструктура на старте это 5–10 тысяч в месяц, VPS плюс домен. GigaChat API при 100 активных пользователях обходится в 3–5 тысяч рублей в месяц. Итого операционные расходы на начальном этапе составляют 8–15 тысяч рублей в месяц. При росте аудитории до 1000 пользователей расходы на API вырастут пропорционально, но серверная часть масштабируется без значительных затрат благодаря облачной инфраструктуре.

2.3.3 Модель монетизации и ценообразование

Подписка по от 299–700 рублей в месяц. Для сравнения, разовая консультация с карьерным коучем стоит 3000–5000 рублей, курс подготовки к собеседованиям от 15 000 рублей, LeetCode Premium 35 долларов в месяц. На этом фоне подписка ViewTrain выглядит доступной альтернативой. Годовая подписка планируется со скидкой 30%, что составит около 5900 рублей за год.

2.3.4 Прогноз окупаемости

Для окупаемости операционных расходов достаточно 20 платящих пользователей. При подписке 700 рублей это 14 000 рублей в месяц, что

покрывает серверы и API. На возврат затрат на разработку при 40–50 подписчиках уйдёт примерно год. При росте базы до 200–300 платящих пользователей месячная выручка выходит на 140–210 тысяч рублей, что позволяет нанять дополнительного разработчика и масштабировать продукт.

Конверсия из бесплатных в платящих оценивается в 5–10% на основе данных аналогичных freemium-продуктов в EdTech. При 1000 зарегистрированных пользователей это 50–100 подписчиков и выручка 35–70 тысяч рублей в месяц.

Помимо затрат на проектирование, важно оценить перспективы монетизации продукта. ViewTrain использует freemium-модель с ежемесячной подпиской. При целевой цене подписки 599 рублей в месяц можно рассчитать базовый сценарий окупаемости. Стоимость привлечения одного пользователя (CAC) на российском EdTech-рынке варьируется от 150 до 500 рублей в зависимости от канала. При использовании контент-маркетинга и органического трафика через профильные сообщества средний CAC для ViewTrain ориентировочно составит 250 рублей. Конверсия из бесплатных пользователей в платных для freemium-продуктов в EdTech составляет в среднем 3–7%. Для расчёта используется консервативная оценка — 5%.

Таблица 3.5 — Прогноз выручки при базовом сценарии (первый год)

Показатель	Значение
Целевая аудитория (зарегистрированные пользователи за год)	5 000
Затраты на привлечение (CAC × 5 000)	1 250 000 руб.
Конверсия в платную подписку (5%)	250 пользователей
Средний срок подписки	4 месяца
Выручка за год (250 × 599 × 4)	598 500 руб
Себестоимость AI-инфраструктуры (GigaChat API, серверы)	~200 000 руб.
Общие затраты (привлечение + инфраструктура + проектирование)	~1 703 000 руб.
Чистый результат первого года	–1 104 500 руб.

Первый год убыточен, что типично для стартапов на стадии MVP. Однако при росте базы пользователей затраты на привлечение снижаются за счёт сарафанного радио и органического трафика, а конверсия растёт по мере улучшения продукта. При базе в 15 000 пользователей на второй год и конверсии 7% прогнозируемая годовая выручка составит около 2 500 000 рублей, что выводит проект на точку безубыточности.

Таблица 3.6 — Прогноз окупаемости (три года)

Год	Пользователи	Конверсия	Платных	Выручка (руб.)	Затраты (руб.)	Результат (руб.)
1	5000	5	250	598500	1703000	−1104500
2	15000	7	1050	2515800	1400000	+1115800
3	30000	8	2400	5750400	1800000	+3950400

Кроме того, выход в B2B-сегмент (корпоративные лицензии для HR-отделов) способен значительно увеличить средний чек. Одна корпоративная лицензия на 50 сотрудников при цене 300 рублей за пользователя в месяц приносит 180 000 рублей в год — эквивалент 300 индивидуальных подписок по месячной конверсии. Таким образом, вложения в проектирование UI/UX экономически обоснованы. Они сокращают затраты на разработку, предотвращают дорогостоящие исправления после запуска и закладывают фундамент для продукта, способного выйти на окупаемость в течение двух лет.

ЗАКЛЮЧЕНИЕ

Работа была посвящена клиентской части платформы ViewTrain, это веб-приложение для подготовки к IT-собеседованиям с AI-интервьюером на базе GigaChat. В первой главе разбирались зарубежные и отечественные исследования по адаптивному обучению и LLM в образовании. Там же анализировались LeetCode, Pramp, InterviewBit и AlgoExpert, ни одна из платформ не закрывает все потребности сразу. Вторая глава про выбор технологий. Методы подготовки анализировались три штуки, ни один сам по себе не работает, их пришлось комбинировать. Фреймворк Vue, AI-модель GigaChat. В третьей главе описано как собирался проект. Каркас поднимался на Vue 3 с Vite, самым сложным оказался модуль чата, потому что ответы от GigaChat идут кусками через WebSocket и их нужно дописывать в реальном времени. Отдельно возился с Monaco Editor, он тяжёлый и без lazy loading тормозил загрузку. Тестировал в Vitest, TTI уложился в 3 секунды. Четвёртая глава про интерфейс. Проектировался под стрессовую ситуацию собеседования, ничего лишнего на экране, все состояния системы видны пользователю. Макеты делались от авторизации до настроек профиля, онбординг в два шага, банк вопросов с подсветкой кода. Все задачи ВКР выполнены. Платформа работает, пользователь может зарегистрироваться, выбрать направление и язык, пройти собеседование с AI и получить обратную связь. Дальше планируется добавить PWA чтобы приложение работало на телефоне, голосовой режим через WebRTC и расширенную аналитику с рекомендациями что повторить.

СПИСОК ЛИТЕРАТУРЫ

[1] Corbett, A. T., Anderson, J. R. Knowledge Tracing: Modeling the Acquisition of Procedural Knowledge // User Modeling and User-Adapted Interaction. — 1994. — Vol. 4, No. 4. — P. 253–278. — DOI: 10.1023/A:1009597714

[2] Piech, C., Bassen, J., Huang, J., Ganguli, S., Sahami, M., Guibas, L., Sohl-Dickstein, J. Deep Knowledge Tracing // Advances in Neural Information Processing Systems 28 (NIPS 2015). — P. 505–513.

[3] Deterding, S., Dixon, D., Khaled, R., Nacke, L. From Game Design Elements to Gamefulness: Defining "Gamification" // Proceedings of the 15th International Academic MindTrek Conference. — 2011. — P. 9–15. — DOI: 10.1145/2181037.2181040.

[4] Kasneci, E., Sessler, K., Küchemann, S. et al. ChatGPT for Good? On Opportunities and Challenges of Large Language Models for Education // Learning and Individual Differences. — 2023. — Vol. 103. — Art. 102274. — DOI: 10.1016/j.lindif.2023.102274.

[5] Mehrabi, N., Morstatter, F., Saxena, N., Lerman, K., Galstyan, A. A Survey on Bias and Fairness in Machine Learning // ACM Computing Surveys. — 2021. — Vol. 54, No. 6. — Art. 115. — P. 1–35. — DOI: 10.1145/3457607.

[6] VanLehn, K. The Relative Effectiveness of Human Tutoring, Intelligent Tutoring Systems, and Other Tutoring Systems // Educational Psychologist. — 2011. — Vol. 46, No. 4. — P. 197–221. — DOI: 10.1080/00461520.2011.611369.

[7] Popenici, S. A. D., Kerr, S. Exploring the Impact of Artificial Intelligence on Teaching and Learning in Higher Education // Research and Practice in Technology Enhanced Learning. — 2017. — Vol. 12, No. 1. — Art. 22. — DOI: 10.1186/s41039-017-0062-8.

[8] W3C Web Content Accessibility Guidelines (WCAG) 2.1. — 2018. — URL: <https://www.w3.org/TR/WCAG21/>

- [9] Vue.js Official Documentation. — URL: <https://vuejs.org/guide/introduction.html>.
- [10] Андреев А. А., Солдаткин В. И. Цифровая педагогика: дидактика цифровой трансформации образования. — М.: Издательство МЭИ, 2021. — 287 с.
- [11] Джумагулова А. Н. Тренды в оценке цифровых компетенций на рынке IT: анализ требований работодателей // Цифровая социология. — 2022. — Т. 5, № 4. — С. 45–56.
- [12] Патаракин Е. Д., Ярмахов Б. Б. Образовательные данные и анализ учебной деятельности (Learning Analytics). — СПб.: Питер, 2020. — 208 с.
- [13] Роберт И. В. Теория и методика информатизации образования. — М.: БИНОМ. Лаборатория знаний, 2020. — 398 с.
- [14] Официальная документация GigaChat API. — URL: <https://developers.sber.ru/docs/ru/gigachat/overview>.
- [15] Официальная документация Vuetify. — URL: <https://vuetifyjs.com/en/getting-started/installation/>.
- [16] Официальная документация Pinia. — URL: <https://pinia.vuejs.org/introduction.html>.

ПРИЛОЖЕНИЯ

Приложение А — Структура файлов проекта

Приложение Б — Ключевые фрагменты кода

Б.1 Конфигурация API-эндпоинтов (config.js)

Б.2 HTTP-клиент с авторизацией (client.js)

Б.3 Модуль аутентификации (auth.js)

Б.4 Компонент чата собеседования (ChatInterview.jsx)

Приложение В — API-контракты

Приложение А — Структура файлов проекта

Ниже представлена структура каталогов клиентской части приложения ViewTrain.

```
src/
├── App.jsx
├── App.css
├── main.jsx
├── index.css
├── api/
│   ├── auth.js
│   ├── client.js
│   ├── config.js
│   ├── directions.js
│   └── languages.js
├── components/
│   ├── Chat.jsx
│   ├── EmailInput.jsx
│   ├── NameInput.jsx
│   ├── PhoneInput.jsx
│   └── LoadingScreen.jsx
├── pages/
│   ├── ChatInterview.jsx
│   ├── ChatInterview.css
│   ├── onboarding-1.jsx
│   ├── Questions.jsx
│   ├── Questions.css
│   ├── Statistics.jsx
│   ├── Statistics.css
│   ├── SettingsPage.jsx
│   └── SettingsPage.css
├── config/
│   └── api.js
```

```
└─ assets/  
  └─ (изображения)
```

Приложение Б — Ключевые фрагменты кода

Б.1 Конфигурация API-эндпоинтов (config.js)

```
export const API_CONFIG = {  
  BASE_URL: '',  
  ENDPOINTS: {  
    AUTH: {  
      REGISTER: '/auth/register',  
      LOGIN: '/auth/login',  
      LOGOUT: '/auth/logout',  
      ME: '/auth/me',  
      CHECK_EMAIL: '/auth/check-email'  
    },  
    INTERVIEW: {  
      START: '/interview/start',  
      SUBMIT_ANSWER: '/interview/submit-answer',  
      COMPLETE: '/interview/complete',  
      HISTORY: '/interview/history'  
    }  
  }  
};
```

Б.2 HTTP-клиент с авторизацией (client.js)

```
class ApiClient {
  constructor() {
    this.baseUrl = API_CONFIG.BASE_URL;
  }

  async request(endpoint, options = {}) {
    const url = `${this.baseUrl}${endpoint}`;
    const headers = {
      'Content-Type': 'application/json',
      ...options.headers,
    };

    const token = localStorage.getItem('token');
    if (token) {
      headers['Authorization'] = `Bearer ${token}`;
    }

    const response = await fetch(url, {
      ...options, headers,
    });

    let data = await response.json();

    if (!response.ok) {
      const error = new Error(data.detail);
      error.status = response.status;
      if (response.status === 401) {
        localStorage.removeItem('token');
      }
      throw error;
    }
    return data;
  }

  async get(endpoint) {
    return this.request(endpoint);
  }

  async post(endpoint, data) {
    return this.request(endpoint, {
      method: 'POST',
      body: JSON.stringify(data),
    });
  }
}
```

Б.3 Модуль аутентификации (auth.js)

```
export const authApi = {
  async register(email, password, name, phone) {
    return apiClient.post(
      API_CONFIG.ENDPOINTS.AUTH.REGISTER,
      { email, password, name, phone }
    );
  },

  async login(email, password) {
    const response = await apiClient.post(
      API_CONFIG.ENDPOINTS.AUTH.LOGIN,
      { email, password }
    );
    if (response.access_token) {
      localStorage.setItem('token',
        response.access_token);
    }
    return response;
  },

  async checkEmailExists(email) {
    const response = await apiClient.post(
      API_CONFIG.ENDPOINTS.AUTH.CHECK_EMAIL,
      { email }
    );
    return response.exists;
  }
};
```

Б.4 Компонент чата собеседования (ChatInterview.jsx)

```
const ChatInterview = () => {
  const [messages, setMessages] = useState([]);
  const [inputMessage, setInputMessage] = useState('');
  const [currentQuestionId, setCurrentQuestionId] = useState(null);
  const [interviewStatus, setInterviewStatus] = useState('not_started');
  const [progress, setProgress] = useState(0);
  const messagesEndRef = useRef(null);

  const scrollToBottom = () => {
    messagesEndRef.current?.scrollIntoView({
      behavior: "smooth"
    });
  };

  useEffect(() => {
    scrollToBottom();
  }, [messages]);

  useEffect(() => {
    startInterview();
    return () => {
      document.body.classList.remove('chat-open');
    };
  }, []);

  const startInterview = async () => {
    const response = await fetch(
      `${API_BASE_URL}/interview/start`,
      { method: 'GET', headers: getAuthHeaders() }
    );
    const data = await response.json();
    setInterviewStatus('ongoing');
    setMessages([
      {
        text: 'Давайте начнем интервью...',
        isBot: true,
        timestamp: new Date()
      }
    ]);
    await getNextQuestion();
  };

  const handleSendMessage = async (e) => {
    e.preventDefault();
    if (!inputMessage.trim()) return;

    const newMessage = {
      text: inputMessage,
      isBot: false,
      timestamp: new Date()
    };
    setMessages(prev => [...prev, newMessage]);
    setInputMessage('');
    // отправка на сервер...
  };
};
```

Б.5 Маршрутизация приложения (App.jsx)

```
function AppWrapper() {  
  return (  
    <Router>  
      <Routes>  
        <Route path="/" element={<App />} />  
        <Route path="/onboarding-1"  
          element={<Onboarding1 />} />  
        <Route path="/chat-interview"  
          element={<ChatInterview />} />  
        <Route path="/statistics"  
          element={<Statistics />} />  
        <Route path="/questions"  
          element={<Questions />} />  
        <Route path="/options"  
          element={<SettingsPage />} />  
      </Routes>  
    </Router>  
  );  
}
```


Приложение В — API-контракты

Взаимодействие клиента с сервером осуществляется по следующим эндпоинтам:

Аутентификация:

POST /auth/register — регистрация нового пользователя

POST /auth/login — авторизация, возвращает JWT-токен

POST /auth/check-email — проверка существования email

POST /auth/logout — выход из системы

GET /auth/me — получение данных текущего пользователя

Онбординг:

GET /auth/directions — список доступных направлений

GET /auth/languages — список доступных языков

POST /auth/select-directions — сохранение выбранных направлений

POST /auth/select-languages — сохранение выбранных языков

Собеседование:

GET /interview/start — запуск новой сессии

GET /interview/question — получение следующего вопроса

POST /interview/submit-answer — отправка ответа

POST /interview/complete — завершение сессии

GET /interview/history — история собеседований